

Service configuration & settings

Early Access · MessageFoundry v0.3.2 · September 2026

Status: the settings catalog is implemented. The `ServiceSettings` model and loader use command-line, environment-variable, file, then default values. See `config/settings.py`. The `serve` command supports `--service-config`.

The model validates every section below except `[engine]`, which has no model. The catalog includes `[store]` for SQLite and server databases, `[api]`, `[inbound]`, and `[delivery]`. Delivery settings supply retry, ordering, and alert defaults when an outbound connection has none. Other sections include `[environments]`, `[logging]`, `[auth]`, `[ai]`, and `[retention]`. The `[environments]` section locates value files. The active environment comes from `[ai].environment`. The retention job enforces retention settings and does SQLite maintenance.

The catalog also includes `[tls]` client trust anchors (ADR 0093), `[reference]`, `[backup]` (ADR 0049), and `[dr]` (ADR 0048).

The loader ignores unimplemented keys without a warning. Each section model uses pydantic `extra="ignore"`. The following entries have no effect: - The complete `[engine]` section. - `[delivery].outbox_workers` and `[delivery].dead_letter`. - `[logging].file`, `[logging].max_bytes`, and `[logging].backups`. - `[retention].audit_days`, which is reserved. Audit records have no expiry by design. - `[reference].max_staleness_seconds` and `[ai].baa_attested`. - `[update_check].index_url` and `[update_check].index_allowed_hosts`.

Principle — two kinds of configuration

MessageFoundry uses two types of configuration:

1. **Message graph.** The interface author writes Connections, Routers, and Handlers in Python (`config/wiring.py`). The engine loads them from `--config`. The graph does not use YAML or declarative channel configuration.
2. **Service settings.** The operator sets the store location, credentials, API address, logging, retention, and retry defaults. This document describes these settings. Keep secrets out of source control.

Mechanism

The `messagefoundry.toml` file contains one section for each settings group. It uses TOML, as does `pyproject.toml`.

Environment variables override file values. Command-line flags override common settings. The order of precedence is:

```
CLI flag > environment variable > messagefoundry.toml > built-in default
```

- File location: `./messagefoundry.toml` by default, or `--service-config <path>`.
- Supply secrets through environment variables or secret references. Do not put plaintext secrets in the file. Environment variables override file values.
- Use `MEFOR_<SECTION>_<KEY>` for environment variables, for example `MEFOR_STORE_PASSWORD` or `MEFOR_API_PORT`. The parser splits at the first `_` after the prefix. It compares the section name with a list of known sections.
- Five sections have no environment-variable support: `[sandbox]`, `[service]`, `[cert_monitor]`, `[secret_rotation]`, and `[update_check]`. The loader ignores their `MEFOR_*` variables without a warning. Set their values in the file.
- The loader creates a typed `ServiceSettings` (pydantic) model at startup. The engine and store read this model. The `serve` flags provide command-line overrides.

Settings catalog

[store] — message store / DB

`StoreSettings` implements these keys. You can select SQLite, Postgres, or SQL Server. SQLite is the default and needs no extra dependencies. Postgres and SQL Server need their optional packages.

The [capability matrix](#) lists the features for each backend.

Key	Type	Default	Notes
<code>backend</code>	enum	<code>sqlite</code>	<code>sqlite</code> · <code>postgres</code> · <code>sqlserver</code> · (later <code>mysql</code> / <code>oracle</code>) — all three implemented; see the capability matrix
<code>path</code>	str	<code>./messagefoundry.db</code>	SQLite only
<code>synchronous</code>	enum	<code>normal</code>	SQLite: <code>normal</code> / <code>full</code>
<code>group_commit_window_ms</code>	float (ms)	<code>0.0</code>	SQLite only (ADR 0055). When <code>> 0</code> , a committer coroutine combines grouped stage mutations into one durable commit. The mutations are <code>enqueue_ingress</code> , <code>route_handoff</code> , <code>transform_handoff</code> , <code>mark_done</code> , <code>complete_with_response</code> , <code>dead_letter_now</code> , and <code>mark_failed</code> . This shares the fsync cost, with a larger benefit under <code>synchronous = full</code> than under <code>normal</code> . A member waits up to this window for other members before commit. Claim, reference-snapshot, and audit writes remain separate. The hash chain must not batch. <code>0</code> disables grouping and preserves the inline-commit behavior. Server databases ignore this setting and combine work through their connection pools.
<code>group_commit_max_batch</code>	int	<code>64</code>	SQLite only . When the batch reaches this size, the committer commits immediately. It does not wait for the rest of <code>group_commit_window_ms</code> . This limits batch size and delay under load. Ignored when <code>group_commit_window_ms = 0</code> .
<code>fifo_claim_batch</code>	int	<code>1</code>	all backends (ADR 0058). Max rows the INGRESS/ROUTED FIFO claim takes per commit. <code>1 = OFF</code> (the workers claim one row per commit — byte-identical to before). <code>> 1</code> (clamped <code>1..64</code>) claims the contiguous due head-prefix in one commit and then processes each row in strict FIFO order with its own off-loop route/transform + separate handoff, amortizing the standalone claim commit toward 1/N. A not-due or producer-locked head still blocks the lane (strict per-lane FIFO, #285). The outbound/delivery claim is never batched. Opt-in throughput tuning (recommend <code>8 - 16</code>); size against worst-case message size, since N decrypted bodies are resident per lane between the claim and the N handoffs.
<code>fifo_claim_fold_reset</code>	bool	<code>false</code>	SQL Server only (ADR 0114 sub-lever C). Folds the pooled claim's session <code>LOCK_TIMEOUT</code> reset into the claim batch on the clean success path at INGRESS/ROUTED (the write-less commit#2 disappears; the shielded finally-guard still runs on every non-clean exit — 1222, kept≠claimed, cancellation, any error). OUTBOUND/RESPONSE are never folded. <code>false</code> = byte-identical shipped batch + guard. Flip only after its own ADR 0114 §8 bench gate (AC-14).
<code>fifo_claim_proc</code>	bool	<code>false</code>	SQL Server only (ADR 0114 sub-lever A). Executes the pooled claim via the two lane-family versioned procs <code>dbo.mefor_claim_fifo_heads_cid_v1</code> / <code>_dst_v1</code> (fixed-arity <code>{CALL}</code> , one JSON lanes parameter) instead of the ~3 KB ad-hoc batch. Needs database <code>COMPATIBILITY_LEVEL >= 130</code> (SQL Server 2016); fails safe to the batch, loudly , when the procs are missing, hand-edited (body-hash mismatch), or compat <code>< 130</code> — never a lane outage. A hardened split-principal deployment must <code>GRANT EXECUTE</code> on both procs to the runtime principal (the bootstrap principal owns them). <code>false</code> = byte-identical. Flip only after its own §8 gate (AC-14).

Key	Type	Default	Notes
<code>fifo_claim_prepared</code>	bool	<code>false</code>	SQL Server only (ADR 0114 sub-lever B). Stabilizes the pooled claim's statement text (one JSON lanes parameter) and retains a prepared claim cursor on store-owned dedicated connections (INGRESS/ROUTED; the non-DDL fallback lane to <code>fifo_claim_proc</code>). Logs + no-ops unless <code>fifo_claim_fold_reset</code> is on (without the fold the finally-guard's reset would evict the one-slot prepare cache every call). <code>false</code> = byte-identical. Flip only after its own \$8 gate (AC-14).
<code>encryption_key</code>	secret	—	env only (<code>MEFOR_STORE_ENCRYPTION_KEY</code>); base64 32-byte active key — when set, PHI columns (<code>raw</code> / <code>payload</code> + <code>error</code> / <code>last_error</code> / <code>detail</code>) are AES-256-GCM-encrypted at rest. Mint one with <code>messagefoundry gen-key</code> . Empty = off. See PHI.md §3 .
<code>encryption_keys_retired</code>	secret	—	env only (<code>MEFOR_STORE_ENCRYPTION_KEYS_RETIRED</code>); comma-separated base64 decrypt-only keys kept available during a rotation until <code>messagefoundry rotate-key</code> finishes re-encrypting under the active key (ASVS 11.2.2).
<code>encryption_key_file</code>	path	—	Windows DPAPI-protected key file (WP-11d, ASVS 13.3.1), created by <code>messagefoundry protect-key</code> . If <code>encryption_key</code> is unset, the store uses <code>CryptUnprotectData</code> to read the active key from this file. The plaintext key does not enter the service environment. Windows only. An environment key takes precedence when both sources are present. This is a path, not a secret, so it can be stored in the configuration file. If unset, the engine uses <code>encryption_key</code> , the cross-platform default.
<code>aad_bind</code>	bool	<code>true</code>	not a secret (<code>MEFOR_STORE_AAD_BIND</code>); cell binding (ASVS 11.3.3, ADR 0019). New at-rest AES-256-GCM writes use the cell-bound <code>mfenc:v2</code> writer — each value is bound to its (<code>table</code> , <code>column</code> , <code>row</code>) cell via GCM Associated Data, so a ciphertext cut-and-pasted into another cell fails the auth tag (dead-lettered <code>CipherError</code>) instead of silently decrypting. On by default (ADR 0148 GIVEN 1: the shipped configuration runs the hardened path). Setting it <code>false</code> selects the frozen <code>mfenc:v1</code> writer (byte-identical at rest) and is a loosening — <code>security_loosenings()</code> names it, so the opt-out is never silent. No effect without an <code>encryption_key</code> (the identity cipher has nothing to bind). Legacy <code>v1</code> rows still decrypt (dual-read); <code>messagefoundry rotate-key</code> upgrades them <code>v1</code> → <code>v2</code> , so the default is safe and reversible on an existing store.
<code>key_provider</code>	enum	<code>auto</code>	selects how the active/retired DEK bytes are <i>sourced</i> — never how they are used (the cipher, keyring, and <code>mfenc:v1</code> format are unchanged; ADR 0019, ASVS 13.3.3). <code>auto</code> (default) is the env-then-DPAPI ladder, byte-identical to the pre-seam behavior; <code>env</code> / <code>dpapi</code> pin a single built-in source; <code>aws_kms</code> · <code>azure_kv</code> · <code>gcp_kms</code> · <code>vault</code> · <code>pkcs11</code> envelope-decrypt a wrapped DEK inside an HSM/KMS/Vault (lazy optional extras — not built yet ; selecting one fails closed at <code>serve</code> , never a silent downgrade). Names a <i>provider</i> , not key material, so it is not a secret.
<code>cipher_provider</code>	enum	<code>aesgcm</code>	selects the at-rest cipher itself — distinct from <code>key_provider</code> , which only <i>sources</i> DEK bytes for the in-process cipher (ADR 0138, ASVS 13.3.3). <code>aesgcm</code> (the default) is that in-process AES-256-GCM cipher, byte-identical to today. <code>vault_transit</code> performs the bulk encrypt/decrypt inside Vault/OpenBao Transit, so the plaintext DEK never enters engine heap; at-rest values then carry the <code>mfenc:v3:</code> marker, the local <code>encryption_key</code> / <code>key_provider</code> go unused (Transit holds the key), and the audit chain is keyed by Transit's <code>generate_hmac</code> — computed inside the vault, so no HMAC key enters heap either. Threaded through all three backends. Vault address / token / data-key name come from <code>MEFOR_STORE_VAULT_ADDR</code> , <code>MEFOR_STORE_VAULT_TOKEN</code> and <code>MEFOR_STORE_TRANSIT_KEY</code> (optionally <code>MEFOR_STORE_TRANSIT_AUDIT_KEY</code>). Names a <i>provider</i> , not key material, so it is not a secret. Any other value fails closed at <code>open_store</code> — never a silent downgrade to plaintext.
<code>require_encryption</code>	bool	<code>false</code>	When <code>true</code> , <code>serve</code> refuses to start without an encryption key in any environment, including synthetic environments. Disabled by default.
<code>allow_unencrypted_phi</code>			→ moved to <code>[security].allow_unencrypted_phi</code> (ADR 0118) — set it there; no longer accepted in <code>[store]</code> .

Key	Type	Default	Notes
<code>server , port</code>	str/int	— / 1433	server DBs (required for <code>sqlserver</code>)
<code>database</code>	str	—	server DBs (required for <code>sqlserver</code>)
<code>auth</code>	enum	<code>sql</code>	<code>sql · integrated · entra</code> (SQL Server). <code>integrated</code> connects <code>Trusted_Connection=yes</code> — the service account's Windows identity authenticates (no SQL password); the turnkey gMSA walkthrough (grant the gMSA a SQL login + run the service under it) is <code>DEPLOY-SERVER-DB.md §1.1</code> .
<code>username</code>	str	—	server DBs (required when <code>auth = sql</code>)
<code>password</code>	secret	—	env only (<code>MEFOR_STORE_PASSWORD</code>)
<code>require_managed_identity</code>	bool	<code>false</code>	delegated-identity precondition (#203, ASVS 13.2.1/13.3.2): when <code>true</code> , <code>serve</code> refuses to start (exit 2) unless the store authenticates via a managed identity — SQL Server <code>auth = integrated / entra</code> . SQLite is exempt; Postgres cannot satisfy it. Off by default. The refuse/warn split is <code>[security].enforcement</code> , not the deployment tier — <code>enforce</code> is the shipped default on <code>dev</code> and <code>staging</code> as much as on <code>prod</code> , so a staging box that turns this on and leaves <code>auth = "sql"</code> is refused , not warned; it downgrades to a warning only under <code>enforcement = warn</code>
<code>encrypt , trust_server_certificate</code>	bool	<code>true / false</code>	TLS to the DB
<code>ssl_root_cert</code>	path	—	Server databases only. Pins the database certificate through a file without importing a private or self-signed CA into machine-wide trust. Requires <code>encrypt = true</code> and <code>trust_server_certificate = false</code> . It never disables verification. PostgreSQL uses an <code>asyncpg SSLContext</code> CA bundle and checks both chain and hostname. SQL Server uses the ODBC Driver 18.1+ <code>ServerCertificate</code> keyword for an exact leaf-certificate match. SQLite rejects this setting because it has no TLS. A missing file prevents configuration load. This is a path, not a secret, and can be stored in configuration. Refer to <code>DEPLOY-SERVER-DB.md §5</code>.
<code>multi_subnet_failover</code>	bool	<code>false</code>	SQL Server only. Adds the ODBC <code>MultiSubnetFailover=Yes</code> keyword for an Always On Availability Group listener across subnets. The driver can connect to the active subnet without waiting for each other subnet to time out. PostgreSQL and SQLite ignore this setting. Disabled by default. Only multi-subnet availability groups need it.
<code>pool_size</code>	int	40	server DBs — server-DB only (no-op on SQLite). The inverted-U optimum (raised from 5; do not set higher — over-provisioning is catastrophic, <code>ADR 0062</code>). Per engine: <code>engines × pool_size</code> share one <code>max_connections</code> — see <code>DEPLOY-SERVER-DB.md §3</code>
<code>connect_timeout , command_timeout</code>	int (s)	15 / 30	server DBs
<code>warm_pool</code>	bool	<code>true</code>	Server databases only. At graph startup or promotion, the engine opens pooled connections in the background. This reduces TCP, TLS, and login delays during connection bursts. The operation is best-effort and releases its resources. SQLite ignores it. Enabled by default. Set <code>false</code> if connection limits or license terms require it.
<code>warm_pool_timeout</code>	num (s)	15	Maximum background warm-up time for server databases. On expiry, the engine logs a warning and uses the partially opened pool. Must be <code>> 0</code> . A clustered node rejects an explicit value <code>>= [cluster].leader_fence_timeout_seconds</code> . The default is 15 seconds, below the 20-second fence timeout.
<code>warm_pool_target</code>	int	—	Number of server-database connections to open in advance. If unset, uses <code>min(pool_size-1, pool_size//2)</code> . An explicit value is limited to <code>pool_size-1</code> . A pool of 1 is not warmed. At <code>pool_size = 40</code> , <code>min(39, 20) = 20</code> connections open per engine at startup.

Key	Type	Default	Notes
<code>db_schema, application_name</code>	str	— / <code>messagefoundry</code>	optional (<code>db_schema</code> $\Rightarrow$ env <code>MEFOR_STORE_DB_SCHEMA</code>)
<code>lease_ttl_seconds</code>	num (s)	60	server DBs — the in-flight row-lease TTL . A worker that claims a row stamps owner + <code>lease_expires_at = now + this</code> ; a renew timer extends it while processing, and the <code>[cluster]</code> leader's reclaim sweep recovers only rows whose lease has expired , so a crashed node's work comes back without stealing a live sibling's in-flight rows. The lease is wall-clock across nodes , so set it comfortably above expected clock skew + the renew interval. A shared server-DB field: SQL Server and SQLite don't lease and ignore it.
<code>uploads_dir</code>	path	—	Off unless set . Enables the opt-in uploaded-logs surface (POST <code>/uploads</code> + the <code>/ui/uploaded-logs/upload</code> delegate, ADR 0134); a filesystem dir for operator-uploaded diagnostic logs. A storage path , not a secret. Unset = no PHI-at-rest upload surface exists. See CONNECTIONS.md §"Uploaded-logs file policy" .
<code>max_upload_bytes</code>	int (bytes)	<code>26214400</code> (25 MiB)	Hard cap on a single uploaded file (<code>ge=1</code> , <code>le=512</code> MiB). Bounds the multipart upload buffer and the offline whole-file split; the global 1 MiB HTTP body cap is raised to this value only on the two upload routes.
<code>max_upload_files_per_user</code>	int	100	Max number of uploaded diagnostic files one uploader may retain at once (ASVS 5.2.4, <code>ge=1</code>). A would-be 101st upload is refused HTTP 409 (<code>upload.reject_quota</code>). Default-on once <code>uploads_dir</code> is set — the control cannot ship disabled.
<code>max_upload_total_bytes_per_user</code>	int (bytes)	<code>262144000</code> (250 MiB)	Max aggregate bytes of uploaded files one uploader may retain (ASVS 5.2.4, <code>ge=1</code>). An upload pushing the uploader's total over the cap is refused HTTP 409 . Default-on.
<code>uploads_retention_days</code>	int (days)	30	Age after which an uploaded file (blob+meta pair) is pruned (ASVS 5.2.4, <code>ge=1</code>) — swept opportunistically at save time and by a periodic task; every prune audited (<code>upload.prune</code> , <code>id</code> + uploader only). Default-on.

For `backend = "sqlserver"`, the loader requires `server` and `database`. It also requires `username` when `auth = "sql"`. SQL Server supports the staged pipeline, response capture, and encryption at rest. Install the `sqlserver` extra (`pip install 'messagefoundry[sqlserver]'`) and Microsoft ODBC Driver 18. The CI service-container job tests a real SQL Server. SQLite remains the default with no extra dependencies.

Per-backend capability matrix

Each row is a `supports_*` flag on the `QueueStore` protocol (`store/base.py`). Each cell shows the value declared by the backend class. The engine refuses unsupported features at startup, before it accepts messages.

Capability flag	SQLite	Postgres	SQL Server
<code>supports_ingest_stage</code>	yes	yes	yes
<code>supports_response_capture</code>	yes	yes	yes
<code>supports_pt_reingress</code>	yes	yes	yes
<code>supports_streaming_attachments</code>	yes	yes	yes
<code>supports_fused_sync_handoff</code>	no	no	yes
<code>supports_reference_sets</code>	yes	yes	yes

Request/response capture (ADR 0013), PT/ `Loopback()` re-ingress, and ADR 0006 reference sets work on all three backends. SQL Server supports `capture_response` and `reingress_to` at full parity since #249. It supports reference snapshots since [BACKLOG #235](#), dated 2026-07-16. CI tests verify this on SQL Server 2022 and 2025.

The reference-set capability gate still applies to future backends. If a backend leaves that capability `False`, a graph with `Reference(...)` fails `messagefoundry check`, startup, reload, and promotion.

One capability differs between backends:

- **supports_fused_sync_handoff** — **SQL Server only**. The fused synchronous handoff twins (ADR 0071 B5) collapse a multi-statement handoff into one executor completion. The profiled wall is aiocdb's per-statement thread crossing, which only SQL Server pays: asyncpg is loop-native and SQLite's handoff lock is loop-affine, so neither has anything to fuse. SQL Server is the *most* capable backend here.

`tests/test_store_capability_matrix.py` compares each table cell with the store-class attribute. If you change or add a flag, update this table in the same commit. Otherwise, the test fails.

[api]

Key	Type	Default	Notes
host			→ moved to <code>[security].local_access_only</code> / <code>listen_address</code> (ADR 0118) — set it there; no longer accepted in <code>[api]</code> .
port	int	8765	
expose_docs	bool	false	serve <code>/docs</code> , <code>/redoc</code> , <code>/openapi.json</code> (off by default — widens surface)
config_reload_roots	list[str]	[]	Additional directories that <code>POST /config/reload</code> can load, besides the startup <code>--config</code> directory. The loader executes Python from these directories. List only trusted, administrator-owned directories, such as an IDE staging directory. A path outside the permitted roots returns 403.
tls_cert_file	str	unset	[BUILT] (WP-13a, ADR 0002) : PEM server-certificate path. Setting it turns on in-process TLS — the API serves <code>https / wss</code> , HSTS engages, and a non-loopback bind is allowed without <code>--allow-insecure-bind</code> .
tls_key_file	str	unset	PEM private-key path. Omit this if the certificate PEM includes the key. Requires <code>tls_cert_file</code> .
tls_key_password	secret	unset	passphrase for an encrypted key — env only (<code>MEFOR_API_TLS_KEY_PASSWORD</code>), never the file.
tls_min_version	str	1.2	minimum negotiated TLS version floor (NIST SP 800-52r2): <code>1.2</code> or <code>1.3</code> .
tls_ciphers	str	unset	Optional OpenSSL cipher string. If unset, uses the interpreter's secure defaults.
tls_client_ca_file	str	unset	CA bundle to require + verify client certs (opt-in mTLS, e.g. the console). Requires <code>tls_cert_file</code> .
tls_client_ca_pin	str	unset	Optional lowercase-hex SHA-256 pin for the CA anchor PEM in <code>tls_client_ca_file</code> . A mismatch prevents load and reload (ASVS 6.7.1). If unset, no pin check applies.
tls_client_cert_identities	map str→str	{}	(#200, ADR 0002) : mTLS client-cert → MessageFoundry principal map. Meaningful only with in-process mTLS (<code>tls_client_ca_file</code> set, so uvicorn <code>CERT_REQUIRED</code> -verifies the peer): a verified peer cert's subject CN / SAN is resolved to an existing username through this allow-list , and that principal's RBAC authorizes the request — a service-to-service identity that carries no bearer token. Keys are the qualified cert name <code>CN:<commonName></code> or <code>SAN:<type>:<value></code> (e.g. <code>SAN:DNS:svc.internal</code>); values are existing usernames. Deny-by-default : an unmapped verified cert — or a spoofed CN not listed here — resolves to no identity and is denied. A structured map, so TOML-only (no env-string form). Empty (the default) disables cert-identity. LIVE, not inert (ADR 0083) . Stock uvicorn does not surface the peer cert to the ASGI scope, so <code>serve</code> swaps in a scope-populating uvicorn HTTP-protocol subclass (<code>messagefoundry/api/tls_client_cert.py</code>) whenever <code>tls_client_ca_file</code> and this map are both set — that shim is what lets a <code>CERT_REQUIRED</code> -verified peer cert reach the resolver. Both conditions are required: set the map without the CA and nothing is verified, so nothing resolves.
tls_client_cert_files	list[str]	[]	ASVS 6.4.5 . PEM copies of inbound service callers' client certificates. The <code>[cert_monitor]</code> scan checks them even when callers stop connecting. Handshake checks can inspect only certificates currently presented. These are peer certificates that the engine verifies, not certificates that it presents. Supply public certificates only, never keys. An empty list disables this check.

Key	Type	Default	Notes
<code>trusted_proxies</code>	list[str]	<code>[]</code>	[BUILT] (WP-15). Proxy addresses trusted for <code>X-Forwarded-For</code> and <code>-Proto</code>, through <code>uvicorn forwarded_allow_ips</code>. Audit and rate-limit checks then use the original client address. An empty list trusts no proxy and uses the direct TCP peer. List only the proxy addresses. Every address within an entry can declare its own source address. A broad entry, such as <code>10.0.0.0/8</code>, therefore permits each workstation in that range to spoof its source. Configuration load rejects <code>"*"</code> and unparseable entries. Without that validation, <code>uvicorn</code> would treat invalid entries as unmatched literals and attribute clients to the proxy.
<code>tls_terminated_upstream</code>	bool	<code>false</code>	[BUILT] (WP-15): declare that a reverse proxy / load balancer terminates TLS in front of the engine. Lets a non-loopback bind satisfy the TLS gate without in-process TLS — but only when <code>trusted_proxies</code> is set (else refused at load).
<code>proxy_intra_service_auth</code>	enum	<code>none</code>	Posture-B operator attestation (#200, ADR 0002) — <i>how</i> the proxy→engine hop is authenticated, so a rogue peer on the internal segment cannot impersonate the proxy. <code>none</code> (the default) is undeclared; declare <code>mtls</code> (the proxy presents a client cert), <code>network</code> (an isolated proxy↔engine segment / host firewall allow-list) or <code>shared_secret</code> (a pre-shared header the proxy injects). Attestation only — the engine enforces nothing at run time; it is the record that the hop was considered. Left undeclared under <code>tls_terminated_upstream</code> on a PHI instance, <code>serve</code> refuses when <code>[security].enforcement = enforce</code> and the bind is non-loopback, and warns otherwise (including the recommended loopback-behind-proxy topology).
<code>proxy_tls_min_version</code>	str	<code>unset</code>	the operator-declared TLS version floor the reverse proxy negotiates with browsers: <code>1.2</code> or <code>1.3</code> (NIST SP 800-52r2) — any other value is refused at load. The engine terminates no browser TLS in Posture-B, so it cannot inspect the proxy's negotiated version (ASVS 11.6.2); this is the attested floor, validated only for coherence. Unset = undeclared, gated exactly like <code>proxy_intra_service_auth</code> above.
<code>proxy_tls_ciphers</code>	str	<code>unset</code>	an optional declared OpenSSL cipher list for that proxy floor. When set it must resolve to forward-secret (EC)DHE suites (ASVS 11.6.2) — the same validator <code>tls_ciphers</code> uses — so a declared floor can't itself name a non-forward-secret key exchange. Unset = no cipher declaration; it is not required to satisfy the Posture-B gate (only <code>proxy_intra_service_auth</code> + <code>proxy_tls_min_version</code> are).
<code>serve_ui</code>			→ moved to <code>[security].serve_web_console</code> (ADR 0118) — set it there; no longer accepted in <code>[api]</code>.
<code>serve_ui_explicit</code>	bool	<code>false</code>	Internal setting. Do not set it directly. The loader sets <code>true</code> when <code>[security].serve_web_console</code> is explicitly supplied with either value. If the console wheel is absent, explicit <code>serve_web_console = true</code> prevents startup. With default-on configuration, the missing wheel produces a warning and JSON-only service. Set <code>[security].serve_web_console</code> instead.
<code>public_origin</code>			→ moved to <code>[security].web_console_public_address</code> (ADR 0118) — set it there; no longer accepted in <code>[api]</code>.
<code>ws_allowed_origins</code>	list[str]	<code>[]</code>	Browser <code>Origin</code> allow-list for the native bearer-token <code>/ws/stats</code> path only. The <code>/ui</code> browser WebSocket uses its cookie and <code>public_origin</code> /Host match instead. These are separate controls.

MLLP-over-TLS uses per-connection `tls` and `tls_*` values on `MLLP(...)` (WP-13b). See [CONNECTIONS.md](#) and [ADR 0002](#). Startup refuses a plaintext MLLP listener outside loopback. On the default posture, `serve --allow-insecure-bind` cannot bypass this refusal. Gate #4's transport-TLS work is complete. Native TOTP MFA is also built for local accounts (WP-14). Use `[security].require_mfa`. The loader rejects the old `[auth].require_mfa` key.

WebAuthn passkeys (WP-14b). See [ADR 0068](#). Install the optional `[webauthn]` extra with `pip install messagefoundry[webauthn]`. Then enroll the user on `/ui/account`. No new `[auth]` setting is required. Without the extra, the page shows a notice instead of an error. The WebAuthn relying-party identity uses the external origin. Set it with `[security].web_console_public_address`. The internal field remains `api.public_origin`, but the loader rejects `[api].public_origin` in the file.

A direct loopback deployment derives the relying party from the request URL. Behind a declared reverse proxy (`tls_terminated_upstream`), the served console requires an explicit origin. Without it, `serve` exits 2. **A later host change invalidates every enrolled passkey**. Passkeys retain their original relying party. The account page marks them "unusable (origin changed)".

Console access outside loopback (L5b, ADR 0068 §8). Two configurations are supported: - Direct in-process TLS with `tls_cert_file` and, if needed, `tls_key_file`. - An upstream terminator with `tls_terminated_upstream = true`, `trusted_proxies = ["<proxy egress IP or CIDR>"]`, and `[security].web_console_public_address`.

Each `trusted_proxies` entry must match the proxy's direct TCP peer address. CIDR ranges are supported. Limit each range to the proxy pool because every included host can forge a forwarded source address. Treat `:::1` and `127.0.0.1` as different addresses. A valid but incorrect entry silently disables forwarded-header rewriting. Audit records and rate limits then use the proxy's IP address. The loader rejects invalid entries and `"*"`. With either protected configuration (`exposure_protected`), the console cookie has `Secure` and the response includes HSTS, regardless of request scheme. The internal security/OFF-LOOPBACK-DEPLOYMENT.md contains the runbook and reverse-proxy mTLS examples. See [SECURITY-DOCS-POLICY.md](#) for access to these documents.

[tls] — outbound client trust anchors

This section sets default client trust anchors for outbound MLLP, DICOM, and FTPS connections. See #190 and [ADR 0093](#). The operating system trust store verifies server certificates by default. You can set a private certificate authority here instead of installing it system-wide or repeating each connection's `tls_ca_file`.

This setting selects trust roots. It does not disable verification or weaken connector refusals for missing authorities, `tls_verify=false`, or cleartext transport. A connection's own `tls_ca_file` takes precedence without changes. Loopback connections are exempt. Without a `[tls]` section, the SSL context is unchanged.

Key	Type	Default	Notes
<code>internal_ca_file</code>	path	—	PEM path to the org's internal CA. Not a secret — a path, like <code>tls_cert_file</code> / <code>forward_tls_ca_file</code> . Unset (default) = no internal anchor; every hop uses the OS trust store.
<code>trust_anchor_mode</code>	enum	<code>system</code>	how <code>internal_ca_file</code> composes with the OS default roots on a non-loopback internal hop. <code>system</code> (default) = OS trust store only , <code>internal_ca_file</code> ignored (byte-identical). <code>augment</code> = OS roots and the internal CA (a mixed public + private estate). <code>pinned</code> = only the internal CA, not the public bundle (a fully-private estate; strictest). <code>pinned without internal_ca_file</code> is refused at load — with nothing to pin it would silently fall back to the full OS trust store, i.e. the operator excludes public roots and gets all of them.

[inbound] — inbound listener defaults

Key	Type	Default	Notes
<code>bind_host</code>	str	<code>127.0.0.1</code>	the default network interface every inbound MLLP/TCP listener binds to. Authors never set a <code>host</code> on an inbound connection (a wiring error if they do) — it's a per-environment operator decision here. Binding <code>0.0.0.0</code> exposes unauthenticated MLLP to the network, so it's deliberate (DEV typically loopback, PROD a specific NIC behind a firewall). A non-loopback bind requires <code>tls=true</code> on each MLLP connection: the \$0 exposed-gate refuses a plaintext off-loopback listener at startup with a <code>WiringError</code> . <code>serve --allow-insecure-bind</code> downgrades that refusal only when the instance is not both enforcing and PHI — and since <code>enforcement = enforce</code> is the default and all three built-in env names (<code>dev</code> / <code>staging</code> / <code>prod</code>) derive PHI, on a stock instance the flag is clamped inert and the bind still fails. A single connection may override this with a per-connection <code>bind_address</code> (and restrict peers with <code>source_ip_allowlist</code>) — MLLP/TCP only; see <code>CONNECTIONS.md</code> .
<code>ack_after</code>	enum	<code>ingest</code>	the default ACK timing every inbound inherits (staged pipeline, <code>ADR 0001</code>). <code>ingest</code> = ACK-on-receipt, once the raw message is durably committed to the ingress stage and before routing/transform/delivery. <code>delivered</code> (defer the ACK until delivery succeeds) is not built — wiring it raises a <code>WiringError</code> , so it fails loud rather than silently ACKing early. A connection's own <code>ack_after=</code> overrides this.
<code>stream_inflight_budget_bytes</code>	int (bytes)	<code>0</code>	aggregate cap on the total bytes of over-threshold message bodies concurrently mid-detach across all inbounds (#149, <code>ADR 0105</code>). A detach that would push the running total over it is refused with backpressure (the message is NAK'd/ <code>ERROR</code> 'd, never accepted-and-dropped), so a burst of very large documents can't exhaust memory. <code>0</code> (default) = unlimited — a <i>single</i> body is still bounded by the per-connection <code>max_message_bytes</code> . Only over-threshold streaming detaches count against it.

[environments] — per-environment graph values (DEV/PROD)

The same graph runs in each environment. Values read through `env("key")` can differ.

Set the required `[ai].environment` name in TOML or with `serve --env <name>` (`ADR 0017`). The name is free-form and has no default. This section only locates the value files.

Key	Type	Default	Notes
<code>dir</code>	str	<code>environments</code>	directory holding <code><env>.toml</code> flat key→value tables for non-secret values, versioned in the repo. Resolved against <code>base_dir</code> (below).
<code>base_dir</code>	str	<code>""</code> (= the working dir)	Anchor <code>dir</code> resolves against. Empty keeps the original behavior (relative to the process working directory). Set it to the config-repo root so env-value resolution no longer depends on where <code>serve</code> was launched. A relative value is taken against the working dir; an absolute value is used as-is — on Windows it must be drive-qualified (<code>C:/repo</code>); a leading-slash <code>/repo</code> is drive-relative and still inherits the launch drive (logged as a warning). Overridable per run with <code>serve --project-root</code> .

- A graph value that differs by environment is authored as `env("acme_adt_host")`; the running instance resolves it from `<base_dir>/<dir>/<active-env>.toml` overlaid by `MEFOR_VALUE-<KEY>` env vars (secrets — never the file; env wins). Keys are `lower_snake_case`.
- Anchoring the value files (`base_dir` / `--project-root`)**. A standalone **config repo** (`ADR 0017`) keeps `environments/` at its root — a *sibling* of the `--config` dir. With the default (empty) `base_dir`, the files resolve relative to the **process working directory**, so a `serve` launched from anywhere but the repo root reads **no** env values (a silent empty table, not an error — the missing values then fail loud only when a connector is built). This bites most under **NSSM**, whose working directory is rarely the repo. Pin the anchor so resolution is launch-independent — in the instance's `messagefoundry.toml`: `toml [environments] base_dir = "C:/srv/acme-config" # the config-repo root; environments/<env>.toml live under it` or per run: `messagefoundry serve --config config --env prod --project-root C:/srv/acme-config` (the flag overrides `[environments].base_dir`; precedence is CLI > env > file > default, like every service setting). The startup log prints the **resolved** `environments/<env>.toml` path so you can confirm where values are read from. Running from the repo root keeps working unchanged (the empty default is the working dir).
- A referenced key that is **undefined for the target environment** makes the engine refuse to load or promote that graph (fail loud) — never a silent blank host. See the env files under `environments/` and `samples/config/IB_ACME_ADT.py` for a worked example.

- **Per-face logic inside a transform:** `env()` is a *deferred reference* resolved only when a **connection** spec is built — using it in a handler is an always-truthy object (a bug). To branch a Router/Handler on the deployment, read the active environment **name** with `current_environment()` (the free-form name, e.g. "prod" / "test", or `None` in a dry-run): `python from messagefoundry import current_environment # Corepoint: If ActiveFace="Test" Then MSH-11.1 = "T" if current_environment() in ("staging", "dev"):` `msg.set("MSH-11.1", "T")` The active environment is a deployment constant, so the read is pure + re-run-safe.

Code sets — reference lookup tables (`codesets/`)

A Router or Handler can use a reference table to translate codes. Examples include Epic diet codes and facility codes. Put the table in a **code set**. Read it with `code_set("name")`.

- **Where.** Files live in `codesets/` **relative to the `--config` dir** — a config bundle carries its own reference tables and they **reload with the graph** (`POST /config/reload`). This is distinct from `environments/` (cwd-level endpoint values for `env()`). A missing `codesets/` dir is fine (no code sets). The code-set **name** is the file's stem (`codesets/epic_diets.csv` → "epic_diets").
- **CSV** (`<name>.csv`) — a header row; the **first column is the lookup key**. One other column → the value is that scalar (`str`); several other columns → the value is a `dict {header: cell}`. A duplicate key is a **load error** (fail load).
- **TOML** (`<name>.toml`) — a flat table `key = value` → `{key: scalar}`; a nested `[key]` table → `{key: {...}}` (mirrors the `environments/<env>.toml` shape).
- **Usage.** Capture once at a module's top level (preferred) or look it up at call time inside a handler — both resolve: `python from messagefoundry import code_set, handler, Send`

```
DIET = code_set("epic_diets") # frozen, read-only mapping; captured at import
```

```
@handler("to_dietary") def handle(msg): msg["ODS-3"] = DIET.get(msg["ODS-3"], "") # .get(key, default) — blank on a miss fac =
code_set("facility_mnemonics").get(msg["MSH-4"]) # call-time lookup also works ... return Send("OB_DIETARY", msg) `` A CodeSet is a
read-only Mapping : cs[key] (raises KeyError naming the set on a miss), cs.get(key, default), key in cs, len(cs), iteration. It is
**frozen** — one instance is shared across transforms, so a handler must never mutate the reference data. - **Load errors.** A
missing code_set("missing") file or malformed CSV/TOML raises WiringError. Duplicate keys also raise this error.
The validate, messagefoundry check, and reload operations report it as they report a missing env() value. The loader does not
substitute an empty table.
```

- **Purity caveat.** The lookup is pure (key in → value out), so it's compatible with the staged pipeline's **pure-re-run** invariant (ADR 0001 / CLAUDE.md §2). The one caveat: a hot-reload that **changes** a table between a run and a crash-re-run can make the re-run derive a different output. That's acceptable for reference data (a code set is deliberately operator-editable, and a reload is an explicit, audited act), but it is the one way a transform's re-run can legitimately differ — note it where you document the transform.
- **Editing — by hand or from the IDE.** A code set is a plain `codesets/<name>.csv` you can edit in any editor, **and** a GUI-manageable artifact (ADR 0033). The VS Code extension opens a **grid editor** (rows × columns of strings — the first column is the lookup key) to **create / edit / rename / delete** a translation table; it shells a new `messagefoundry codeset` CLI that owns validation and the atomic write. Both editors write the same file (CSV-first), so a hand edit and a GUI save are interchangeable — mirroring the connections editor (ADR 0007).
- `messagefoundry codeset list --config DIR` — summarize every set under `codesets/` (`.csv` **and** `.toml` ; TOML sets are summarized and shown **read-only** in the grid — TOML-in-grid editing is a fast-follow).
- `messagefoundry codeset show --config DIR --name N` — the grid (headers + rows).
- `messagefoundry codeset upsert --config DIR --data '{...}'` — validate → write `codesets/N.csv` atomically (temp + replace, owner-only perms) → **re-load the written file as the final check**; a bad save rolls back, so the CLI never leaves an unloadable table.
- `messagefoundry codeset rename --config DIR --name N --to M / ... remove --config DIR --name N.`

The CLI is **offline** (no engine start, no egress check — a code set is standalone data); it validates against the **same loader** that runs at startup, and the operator-supplied **name is treated as untrusted data** (rejecting path separators, `..`, absolute/drive paths, and an embedded extension, so a name can't escape `codesets/`). Apply a change with the existing audited promote/reload below. - **Apply changes through reload.** A file edit does not change the running graph. Use `POST /config/reload` (the IDE promote action) to apply it, as for connection or Handler changes. A renamed or removed code set can break a Handler reference. A remaining `code_set("old_name")` call then raises an

error at runtime and gives that message an `ERROR` disposition. The `validate` command only confirms that files parse. It does not detect a missing reference inside a Handler. After a rename or removal, run `messagefoundry check` before promotion. Its dry-run executes transforms and detects broken `code_set(...)` lookups. See [docs/CODESETS.md](#) for the grid editor and reload workflow.

Transform state — cross-message correlation (ADR 0005)

Where code sets are **read-only** reference data, **transform state** is **read/write** correlation data a Handler accumulates across messages: an anonymous-patient mapping (persist a real MRN → a stable anonymized id and reuse it on later messages), order↔result correlation, running aggregates. It is authored against two surfaces from `messagefoundry`:

```
from messagefoundry import handler, Send, SetState, state_get

@handler("anonymize")
def anonymize(msg):
    mrn = msg["PID-3.1"]
    anon = state_get("patient_anon", mrn)          # synchronous read; None on a miss
    ops = []
    if anon is None:
        anon = derive_anon_id(mrn)                # deterministic derivation preferred (see below)
        ops.append(SetState("patient_anon", mrn, anon))
    msg["PID-3.1"] = anon
    return [Send("OB_DOWNSTREAM", msg), *ops]      # Sends and SetStates, mixed in one list
```

- **State writes.** A Handler returns `Send` | `SetState` | `list[Send | SetState]` | `None`. It does not change state directly. Each `SetState(namespace, key, value)` declares an upsert by `(namespace, key)`. The value must be JSON-serializable. Construction validates this requirement. The engine applies the upsert inside the routed-to-outbound handoff transaction. Handlers that return only `Send` objects are unchanged.
- **Exactly-once writes.** State writes and outbound rows commit in the same transaction. A crash before commit leaves no state write. The committed attempt applies each write exactly once per message. This preserves the pure-re-run invariant (ADR 0001, CLAUDE.md §2). A random anonymous ID is safe because only the committed attempt persists. Prefer deterministic values when identity must remain consistent across runs.
- **State reads.** `state_get(namespace, key, default=None)` reads a synchronous in-memory cache. The engine loads the cache at startup and updates it after writes commit. It makes the cache available for each router or transform run, as with `code_set()`. A missing key returns `default`. A read reflects committed state at invocation but is not linearized with a concurrent sibling Handler's write. This suits read-mostly correlation. Race-sensitive read-modify-write operations in one namespace need additional care from the author.
- **Encryption.** State values can contain PHI, such as MRN-to-ID mappings. The store cipher encrypts them with AES-256-GCM, as for `messages.raw`. The `messagefoundry rotate-key` command also rotates state encryption keys.
- **Retention.** Set `[retention].state_max_age_days` to remove stale entries by age. This policy applies globally. Per-namespace retention is planned. Retention is off by default, so entries remain indefinitely. The whole-table cache requires bounded state. Support for unbounded state is planned (ADR 0005).
- **SQL Server.** The `transform_handoff` operation writes state on SQL Server, as on SQLite and Postgres. The cache refreshes after commit. Cross-node state convergence does not apply to the single-node backend described here.

The IDE Test Bench, dry-run, and `messagefoundry check` also resolve `state_get`. Each simulated message gets a fresh in-memory view that collects its declared writes. A later Handler can read an earlier Handler's `SetState` from the same run. The `dryrun` output lists declared state operations only with `--show-phi`, as for message bodies.

Reference sets — external-data enrichment (ADR 0006)

A **code set** is a static table in the configuration bundle. **Transform state** stores read/write correlation data. A **reference set** contains external data, such as a provider directory or database translation table. This corresponds to the Corepoint Data Point or DB Association pattern.

The engine periodically copies the source into a versioned, encrypted store snapshot. A Handler reads this snapshot without an external call. The pure-re-run invariant holds unless a snapshot changes between the original run and a rerun after a crash. This is the same accepted exception as a code-set reload.

- **Declare** a set in a wiring module (registers it into the graph, like `inbound`): `python from messagefoundry import Reference, FileRef, env, handler, Send, reference`

```
Reference("provider_npi", source=FileRef(path=env("provider_npi_csv")), refresh_seconds=3600)
```

`@handler("enrich") def enrich(msg): npi = reference("provider_npi").get(msg["PV1-7.1"]) # pure dict lookup, no I/O if npi: msg.set("PV1-7.13", npi) return Send("OB_DOWNSTREAM", msg)`

`-- **Read a set.** reference(name) returns a frozen, read-only ReferenceSet . Supported operations include rs[k] , rs.get(k, d) , and k in rs . A missing key returns the default. A missing or unsynchronized set raises an error and gives the message an ERROR disposition. Call reference(name) inside a Handler or Router. Do not call it at module level: the snapshot exists only after the store opens and synchronizes. -- **File sources.** FileRef(path=..., encoding=...) reads local CSV or TOML files in code-set format. The engine rereads the file on the refresh schedule. The path can use env() for an externally produced export. -- **Database sources.** DatabaseRef(server=..., database=..., statement=..., key_column=..., value_column=...) runs a read-only SQL query on the refresh schedule. SQL Server needs the supported [sqlserver] extra. Supply secrets through env() . The [egress].allowed_db allowlist controls the connection and refuses unapproved destinations. The key_column is the lookup key. If set, value_column supplies the value. Otherwise, the value is a dictionary of other columns. -- **Synchronization.** ReferenceSyncRunner loads each set at startup before listeners accept messages. It refreshes each set every refresh_seconds . If a source fails, the engine logs the failure, raises an alert, and keeps the last good snapshot. It does not attempt a snapshot write. Other sources and message processing continue. -- **Storage.** Snapshot values can contain PHI. AES-GCM encryption and key rotation protect them, as for state and message bodies. SQLite, Postgres, and SQL Server support snapshot storage ([BACKLOG #235](https://github.com/MEFORORG/MessageFoundry/blob/main/docs/BACKLOG.md), 2026-07-16). The [egress].allowed_db allowlist also applies to DatabaseRef connections. -- **Settings.** [[reference]](#reference) controls the refresh schedule and startup behavior. -- **Dry-run.** A dry-run or check attempts to resolve file-backed sets with literal paths. It does not resolve database-backed sets or paths that use env().`

[reference]

The `ReferenceSyncRunner` implements these settings for reference sets (ADR 0006, Tier 1). See `pipeline/reference_sync.py`. If the graph declares no reference sets, the runner does nothing.

Key	Type	Default	Notes
<code>refresh_interval_seconds</code>	float (s)	<code>3600</code>	Interval for the synchronization loop. Each reference set refreshes when its own <code>refresh_seconds</code> becomes due. Must be <code>> 0</code> . Other values fail configuration load.
<code>sync_on_startup</code>	bool	<code>true</code>	Materialize each declared reference set before inbound listeners start. This makes <code>reference(...)</code> available for the first message. Keep this enabled.
<code>max_staleness_seconds</code>	float (s)	<code>0</code>	Reserved and not enforced. Intended to alert or refuse when the active snapshot exceeds this age. Accepted for future configuration compatibility. <code>0</code> disables it. Must be <code>>= 0</code> .

[auth] — authentication & RBAC

Authentication is required by default. See `SECURITY.md`. Supply the AD bind password through `MEFOR_AUTH_AD_BIND_PASSWORD`. Do not put it in the file.

The `*_rate_limit*` settings limit resource use. The internal security/THREAT-MODEL.md lists the affected functions and those that remain unbounded. See its Resource-demanding functionality section (ASVS 15.1.3) and `SECURITY-DOCS-POLICY.md`.

Key	Type	Default	Notes
<code>enabled</code>			→ moved to <code>[security].require_sign_in</code> (ADR 0118) — set it there; no longer accepted in <code>[auth]</code> .
<code>session_idle_timeout_minutes</code>			→ moved to <code>[security].sign_out_after_idle_minutes</code> (ADR 0118) — set it there; no longer accepted in <code>[auth]</code> .
<code>session_absolute_hours</code>			→ moved to <code>[security].max_session_hours</code> (ADR 0118) — set it there; no longer accepted in <code>[auth]</code> .

Key	Type	Default	Notes
<code>max_sessions_per_user</code>	int	5	Maximum concurrent sessions per user (ASVS 7.1.2). <code>0</code> means unlimited. A login above the limit revokes the user's oldest active session.
<code>step_up_max_age_seconds</code>	int	300	Credential re-verification window for sensitive operations (ASVS 7.5.3). Login or <code>POST /me/reauth</code> must verify the credential within this many seconds. The initial login counts as the first verification.
<code>require_action_step_up</code>	bool	<code>true</code>	action-bound step-up (ADR 0077; ASVS 7.5.1/8.2.4). On by default: the durable-takeover JSON routes — TOTP enroll/confirm and disable-MFA — require a fresh proof bound to that specific action (<code>POST /me/reauth</code> with a matching <code>purpose</code> , single-use) instead of riding the session-wide <code>step_up_max_age_seconds</code> window. It closes the most-exploitable default: a session hijacked inside the 300 s login-seeded window could otherwise bind an attacker's authenticator with no fresh proof. It changes only those factor-binding routes — the broad admin / replay / config / purge routes keep the session-window step-up. <code>false</code> reverts to the legacy session-window behaviour (0.2.x semantics), the documented org opt-out
<code>password_min_length</code>	int	15	local-password policy — ASVS 5.0-aligned, length-first
<code>password_require_uppercase</code> / <code>password_require_lowercase</code> / <code>password_require_digit</code> / <code>password_require_symbol</code>	bool	<code>false</code>	character classes — opt-in , each independently (ASVS 5.0 forbids mandatory composition); turn one on only for a legacy standard that still mandates it
<code>password_check_breached</code>	bool	<code>true</code>	reject known common/breached passwords against a bundled offline top-10k list (no live HIBP call)
<code>password_check_context</code>	bool	<code>true</code>	reject passwords containing app/vendor/HL7 terms (e.g. <code>messagefoundry</code> , <code>me4or</code> , <code>hl7</code> , <code>corepoint</code>)
<code>password_check_username</code>	bool	<code>true</code>	reject a password containing the user's own username (ASVS 6.2.11)
<code>password_breach_corpus_file</code>	path	—	Optional larger offline password corpus that extends the bundled top-10k list (ASVS 6.2.12). Accepts plaintext passwords or HIBP-style SHA-1 <code>HASH[:count]</code> lines, detected automatically. No live HIBP request occurs. Use a curated subset because the engine loads it into memory. Do not load the full ~40 GB HIBP set. This is a path, not a secret.
<code>lockout_threshold</code>	int	5	failed logins before lock (per account)
<code>lockout_minutes</code>	int	15	lockout duration
<code>bootstrap_expiry_hours</code>	int	72	The engine disables the bootstrap administrator once a second administrator exists. An unclaimed account also expires this many hours after creation. Unclaimed means its password was never changed. <code>0</code> disables time-based expiry.
<code>bootstrap_warn_hours</code>	int	24	(ASVS 6.4.5): how long <i>before</i> that deadline to remind an operator that the still-unclaimed bootstrap credential is about to be retired — a <code>bootstrap_admin_expiring [alerts]</code> event, raised once while <code>now</code> is inside <code>[expiry - this, expiry]</code> . Advisory only (it disables nothing); meaningful only when <code>bootstrap_expiry_hours > 0</code> . The deadline itself is also written into <code>bootstrap-admin.txt</code> at issuance.

Key	Type	Default	Notes
<code>initial_password_expiry_hours</code>	int	72	ASVS 6.4.1. An unclaimed administrator-issued initial or reset password expires this many hours after issue. The expiry uses <code>password_changed_at</code> for credentials marked <code>must_change_password</code> . Users with <code>must_change_password = false</code> are unaffected. The bootstrap administrator uses <code>bootstrap_expiry_hours</code> instead. <code>0</code> disables expiry and is not recommended for PHI instances. Without expiry, an unused temporary password can permit authentication and a password change indefinitely.
<code>login_rate_limit_enabled</code>	bool	<code>true</code>	in-process sliding-window limiter on the sign-in surface — <code>/auth/login</code> , <code>/auth/negotiate</code> , <code>/auth/mfa-verify</code> plus the four console entry routes (<code>POST /ui/login</code> , <code>GET /ui/sso</code> , <code>GET /ui/oidc/start</code> , <code>GET /ui/oidc/callback</code>) — in front of the per-account lockout. The same flag also constructs the per-actor credential-ceremony limiter covering <code>/me/password</code> , <code>/me/reauth</code> , <code>/me/mfa/confirm</code> (+ the console re-auth routes); turning it off removes both (see SECURITY.md "Route → limiter map").
<code>login_rate_limit_per_ip</code>	int	10	Maximum attempts per client IP per window. <code>0</code> disables this limit. This value also sets the per-actor credential-ceremony budget for <code>/me/password</code> , <code>/me/reauth</code> , <code>/me/mfa/confirm</code> , and console re-authentication routes. The <code>_per_ip</code> name is historical. Changing the value changes both limiters.
<code>login_rate_limit_global</code>	int	60	Maximum attempts across all clients per sign-in window. <code>0</code> disables this limit. The credential-ceremony limiter has no global limit (<code>glob=0</code>).
<code>login_rate_limit_window_seconds</code>	float	60	Window length shared by the sign-in limiter and per-actor credential-ceremony limiter. Both also use <code>login_rate_limit_per_ip</code> .
<code>phi_read_rate_limit_enabled</code>	bool	<code>true</code>	per-actor anti-automation throttle (ASVS 2.4.1) — bounds scripted PHI harvesting on top of pagination + access auditing. Charged on 7 JSON routes via <code>require_phi_read</code> , on the 4 bulk-PHI step-up GETs at admission (<code>/messages/search</code> , <code>/messages/export</code> , <code>/uploads/{file_id}/messages</code> , <code>/search/layered</code> — <code>require_step_up</code> paces NON-GET only, so these charge it themselves), and on the 5 /ui PHI views via <code>require_ui(..., phi=True)</code>
<code>phi_read_rate_limit_per_actor</code>	int	120	Maximum PHI reads per user per window. The default permits normal console use. <code>0</code> disables this limit.
<code>phi_read_rate_limit_global</code>	int	0	max PHI reads across all users per window (<code>0</code> = off)
<code>phi_read_rate_limit_window_seconds</code>	float	60	sliding-window length
<code>admin_write_rate_limit_enabled</code>	bool	<code>true</code>	per-actor anti-automation pacing on the state-changing admin surface (ASVS 2.4.2) — NON-GET only , charged from one per-actor bucket by both <code>require_step_up</code> and <code>require_paced</code> . JSON API only: no <code>/ui</code> route charges it today (BACKLOG #287)
<code>admin_write_rate_limit_per_actor</code>	int	12	Maximum state-changing administrator writes per actor per window. <code>0</code> disables this limit. There is no global limit, so one operator's work cannot throttle another.
<code>admin_write_rate_limit_window_seconds</code>	float	1.0	Window length. Requests above the budget receive <code>429</code> and <code>Retry-After: 1</code> before further processing.
<code>notify_security_events</code>	bool	<code>true</code>	email the affected user on logout / first-success-after-failures / password-email-role-disable changes (ASVS 6.3.5/6.3.7). Reuses the <code>[alerts]</code> SMTP transport, sent to the user's own address; no SMTP configured → email skipped. The <code>GET /me/security-events</code> feed (over the audit log) is always available regardless of this toggle. On a PHI production instance this push must be <i>effective</i> — see <code>[alerts].security_notifications_required</code> (BACKLOG #188).

Key	Type	Default	Notes
<code>require_mfa</code>			→ moved to <code>[security].require_mfa</code> (ADR 0118) — set it there; no longer accepted in <code>[auth]</code> .
<code>require_mfa_scope</code>			→ set it as <code>[security].require_mfa_scope</code> (ADR 0118) — like its <code>require_mfa</code> sibling it is rejected in <code>[auth]</code> .
<code>totp_skew_steps</code>	int	<code>0</code>	TOTP clock-skew tolerance in 30 s steps applied at verify time (BACKLOG #187, ASVS 6.5.5). Default <code>0</code> = STRICT: only the current 30 s step verifies (tightest replay window — a captured code is valid at most for the rest of its own step). Set <code>1</code> (or <code>2</code>) — the documented opt-out — to restore RFC-6238 network-delay / clock-drift tolerance (<code>1</code> also accepts the immediately-prior and the fast-clock-clamped next step, i.e. the historical ± 1 behaviour; the forward step is clamped to the current step so it never advances the single-use high-water mark). Range 0–2.
<code>mfa_recovery_code_count</code>	int	10	single-use recovery codes minted at TOTP enrollment (the lost-authenticator escape hatch; <code>0</code> disables them, leaving an admin reset as the only recovery path). Range 0–50.
<code>admin_new_ip_step_up</code>	bool	<code>false</code>	admin-interface contextual-risk signal (WP-L3-13, ASVS 8.4.2): when on, a step-up (sensitive admin) request from a client IP the session has not verified from emits an <code>auth.admin_action_new_ip</code> audit + notice and forces a fresh step-up (a re-verify from that address clears it). Advisory + step-up-forcing only — never changes an RBAC decision, never blocks the non-admin path; the audit + notice fire once per (session, new address). Off by default (byte-identical on loopback — <code>127.0.0.1</code> and <code>::1</code> are treated as one host); recommended on for an off-loopback admin deployment. See SECURITY.md "Administrative-interface defense-in-depth".
<code>ad_enabled</code>	bool	<code>false</code>	turn on Active Directory login
<code>ad_server</code>	str	—	e.g. <code>ldaps://dc1.example.com:636</code> (required when <code>ad_enabled</code>)
<code>ad_domain</code>	str	—	UPN suffix, e.g. <code>example.com</code>
<code>ad_user_search_base</code>	str	—	required when <code>ad_enabled</code>
<code>ad_group_search_base</code>	str	—	base for nested-group resolution
<code>ad_bind_dn</code>	str	—	service-account DN used for lookups
<code>ad_bind_password</code>	secret	—	env only (<code>MEFOR_AUTH_AD_BIND_PASSWORD</code>), or use <code>ad_bind_password_secret</code>
<code>ad_bind_password_secret</code>	str	—	connector <code>SecretProvider</code> reference (ADR 0019 §5) — when set and <code>[secrets].provider</code> is configured, the bind password is resolved from that backend (e.g. a Vault KV <code>path#field</code>) instead of <code>ad_bind_password</code> . A reference, not a secret.
<code>ad_use_nested_groups</code>	bool	<code>true</code>	resolve nested groups (<code>LDAP_MATCHING_RULE_IN_CHAIN</code>)
<code>ad_tls_verify</code>	bool	<code>true</code>	validate the LDAPS certificate
<code>ad_tls_ca_cert_file</code>	str	—	trust an internal CA for LDAPS without disabling verification
<code>ad_tls_ca_cert_pin</code>	str	—	Optional lowercase-hex SHA-256 pin for the CA anchor PEM in <code>ad_tls_ca_cert_file</code> . A mismatch prevents load and reload (ASVS 6.7.1). If unset, no pin check applies.
<code>ad_allow_insecure_ldap</code>	bool	<code>false</code>	explicit opt-in to a non- <code>ldaps://</code> bind (trusted-network dev only)

Key	Type	Default	Notes
<code>ad_connect_timeout</code>	float	<code>10.0</code>	Maximum seconds for LDAP/LDAPS TCP connection setup on each <code>ldap3 Server</code> (ASVS 13.1.3). Must be finite and <code>> 0</code> . Configuration load rejects <code>0</code> , negative values, <code>inf</code> , and <code>NaN</code> . The <code>ldap3</code> default, <code>None</code> , waits indefinitely and can hold a worker when a domain controller does not respond.
<code>ad_receive_timeout</code>	float	<code>10.0</code>	Maximum seconds for each LDAP response read, including both binds and every search on each <code>ldap3 Connection</code> . Must be finite and positive.
<code>ad_session_recheck_seconds</code>	int	<code>300</code>	Directory session reconciliation (ADR 0079 mechanism 2). How often to re-resolve directory principals holding live sessions and revoke those AD has disabled or deleted — without it, an AD disable does not take effect until the <code>[security].max_session_hours</code> cap (12 h). 300 (five minutes) is the default (ADR 0148 GIVEN 1 — the hardened path is the shipped path), floored at 60 s (a pass costs one LDAP bind per signed-in directory user). <code>0</code> disables the loop and is a loosening once AD is on — <code>security_loosenings()</code> names it. The default is inert without AD (<code>should_reconcile()</code> also needs an LDAP client), so a non-AD deployment is unaffected; an explicit non-zero value without <code>ad_enabled</code> is still refused rather than left silently dead.
<code>ad_session_recheck_strikes</code>	int	<code>2</code>	Consecutive failed directory lookups before session revocation. Disabled accounts, deleted accounts, and unmatched searches return the same lookup result. The threshold limits revocation from ambiguous results. Range: 1–10.
<code>ad_session_recheck_max_users</code>	int	<code>200</code>	Maximum users checked per pass. Later passes check remaining users, starting with the least recently checked. Large deployments can therefore have longer effective intervals.
<code>ad_session_revoke_max</code>	int	<code>5</code>	Mass-revoke circuit breaker , absolute half. A bad search base / moved OU / service account that lost read rights answers "not found" for <i>every</i> user — indistinguishable from "everyone was disabled".
<code>ad_session_revoke_max_fraction</code>	float	<code>0.34</code>	Proportional threshold for the reconciliation circuit breaker. If a pass exceeds both thresholds, it revokes no sessions, logs ERROR, and writes <code>auth.ad_reconcile_aborted</code> . Both conditions must hold, so changes of 3-of-3 or 50-of-300 still apply. Range: >0.0–1.0. <code>1.0</code> disables the proportional threshold.
<code>kerberos_enabled</code>	bool	<code>false</code>	Windows SSO (experimental, not yet available ; needs <code>ad_enabled</code>)
<code>kerberos_spn</code>	str	—	service principal, e.g. <code>HTTP/host.example.com</code>
<code>oidc_enabled</code>	bool	<code>false</code>	Federated SSO — OIDC auth-code + PKCE relying party (ADR 0142). A third login for an identity that already exists in on-prem AD (needs <code>ad_enabled</code> ; roles come from LDAP, not the token). Off = byte-identical. Needs <code>[security].web_console_public_address</code> (the redirect origin).
<code>oidc_issuer</code>	str	—	https; exact-matched against the <code>id_token iss</code>
<code>oidc_client_id</code>	str	—	also the required <code>aud</code> / <code>azp</code>
<code>oidc_client_secret</code>	str	—	confidential-client secret — env only (<code>MEFOR_AUTH_OIDC_CLIENT_SECRET</code>), never the file
<code>oidc_client_secret_ref</code>	str	—	alternative: a <code>[secrets].provider</code> reference (<code>_ref</code> , not <code>_secret</code> , to avoid <code>oidc_client_secret_secret</code>)
<code>oidc_authorization_endpoint</code> / <code>oidc_token_endpoint</code> / <code>oidc_jwks_uri</code>	str	—	https, operator-pinned (no <code>.well-known</code> discovery)
<code>oidc_allowed_endpoints</code>	list[str]	<code>[]</code>	defence-in-depth host allow-list; refused empty when enabled ; every OIDC endpoint host must be listed

Key	Type	Default	Notes
<code>oidc_tls_ca_cert_file</code>	str	—	the engine's back-channel TLS trust for the IdP (OpenSSL default trust ignores the Windows machine store)
<code>oidc_tls_ca_cert_pin</code>	str	—	Optional lowercase-hex SHA-256 pin for the CA anchor PEM in <code>oidc_tls_ca_cert_file</code> . A mismatch prevents load and reload (ASVS 6.7.1). If unset, no pin check applies.
<code>oidc_redirect_path</code>	str	<code>/ui/oidc/callback</code>	joined to <code>web_console_public_address</code> for the redirect URI
<code>oidc_scopes</code>	list[str]	<code>["openid", "profile"]</code>	no <code>email</code> , no <code>offline_access</code>
<code>oidc_signing_algorithms</code>	list[str]	<code>["RS256"]</code>	coerced through the closed JWS algorithm enum
<code>oidc_username_claim</code> / <code>oidc_username_strip_domain</code>	str / bool	<code>preferred_username</code> / <code>true</code>	strip at <code>@</code> → sAMAccountName
<code>oidc_allowed_username_domains</code>	list[str]	<code>[]</code>	the control that stops a federated principal choosing which on-prem account it resolves to. <code>preferred_username</code> is neither unique nor stable (OIDC Core §5.7) and is operator- or even self-editable on many IdPs, so without this a guest presenting <code>Administrator@attacker.example</code> strips to <code>Administrator</code> and signs in as the on-prem Domain Admin. When <code>oidc_username_strip_domain</code> is on, the claim's UPN suffix must match one of these. Empty falls back to <code>ad_domain</code> ; with neither set, <code>oidc_enabled</code> is refused at load rather than stripping unchecked. List the alternate UPN suffixes of a multi-domain forest here
<code>oidc_clock_skew_seconds</code>	int	<code>60</code>	wall-clock skew tolerance (0–300)
<code>oidc_require_mfa_claim</code>	bool	<code>true</code>	#99(g) control — refuse a token with no configured <code>amr</code> / <code>acr</code> . The engine verifies what the IdP asserts , not what it enforced
<code>oidc_mfa_amr_values</code> / <code>oidc_required_acr_values</code>	list[str]	<code>["mfa"]</code> / <code>[]</code>	either family satisfies the gate; both empty with the gate on is refused
<code>oidc_acr_values</code> / <code>oidc_prompt</code>	str	—	requested authorize params
<code>oidc_jwks_ttl_seconds</code> / <code>oidc_jwks_min_refetch_seconds</code>	int	<code>3600</code> / <code>300</code>	the JWKS cache TTL + the amplification (min-refetch) bound
<code>oidc_flow_ttl_seconds</code> / <code>oidc_flow_cache_max</code>	int	<code>300</code> / <code>512</code>	pending-flow TTL + the reject-when-full bound
<code>oidc_session_max_hours</code>	int	—	caps the federated session below <code>id_token.exp</code> if a tighter bound is wanted (ADR 0079 mechanism 1)

AD-group→role mappings live in the DB and are managed by an admin (`PUT /ad-group-map` or the console Users page), not in this file. Federated logins reuse the **same** AD-group→role mapping — the role source is on-prem AD, never a token claim (ADR 0142).

[ai] — AI coding assistance policy

These settings control the IDE AI assistant. See `AI.md`. The policy is centrally managed and restricted by the security posture. It uses `mode`, `data_scope`, and the active environment name and posture (`environment`, `data_class`, `production`).

Under `mode = managed_endpoint`, the engine broker uses `provider`, `model`, `endpoint`, `api_key`, and `allowed_endpoints`. See ADR 0135. The loader accepts but ignores `baa_attested`.

Key	Type	Default	Notes
<code>mode</code>	enum	<code>byo</code>	<code>off</code> · <code>byo</code> · <code>managed_endpoint</code> · <code>managed_claude</code> · <code>managed_claude_baa</code> . managed_endpoint is built — the engine brokers one <code>code_only</code> prompt to a customer-managed / self-hosted LLM over <code>POST /ai/chat</code> , audited per use (ADR 0135); it never reaches <code>phi</code> scope. <code>managed_claude</code> / <code>managed_claude_baa</code> are future — not serviceable by the current IDE
<code>data_scope</code>	enum	<code>code_only</code>	<code>code_only</code> · <code>synthetic</code> · <code>deidentified</code> · <code>phi</code> , least→most sensitive; capped by <code>production</code> posture and by <code>mode</code> (only <code>managed_claude_baa</code> reaches <code>phi</code>)
<code>environment</code>	str	—	free-form active-environment name (ADR 0017); selects <code>environments/<name>.toml</code> + <code>current_environment()</code> . Required for <code>serve</code> (no default)
<code>data_class</code>			→ moved to <code>[security].handles_real_patient_data</code> (ADR 0118) — set it there; no longer accepted in <code>[ai]</code> .
<code>production</code>			→ moved to <code>[security].production_instance</code> (ADR 0118) — set it there; no longer accepted in <code>[ai]</code> .
<code>provider</code>	str	<code>claude</code>	the broker's request shape; read under <code>mode = managed_endpoint</code> (and recorded in the per-use audit)
<code>model</code>	str	<code>claude-opus-4-8</code>	the model the broker asks for; read under <code>mode = managed_endpoint</code> (also echoed on the reply)
<code>baa_attested</code>	bool	<code>false</code>	accepted-but-ignored — an operator attestation carried for the future <code>managed_claude_baa</code> path
<code>endpoint</code>	str	—	the customer-managed LLM URL. Required for <code>mode = managed_endpoint</code> ; <code>http/https</code> only, and a cleartext <code>http</code> endpoint is refused (it would expose <code>api_key</code>)
<code>api_key</code>	secret	—	the LLM credential — env only (<code>MEFOR_AI_API_KEY</code>), never the file. Required for <code>mode = managed_endpoint</code>
<code>allowed_endpoints</code>	list	<code>[]</code>	fail-closed SSRF allow-list for <code>endpoint</code> 's host; each entry is <code>host</code> (any port) or <code>host:port</code> . An empty list permits nothing — deliberately independent of <code>[egress].allowed_http</code> , which is permissive when empty and so can't gate this surface

Only `code_only` context is ever sent in the MVP (graph names + active editor code) — **never message bodies**. The full resolution/clamping algorithm, the `GET /ai/policy` endpoint, the `messagefoundry ai-policy` CLI, and the `ai:assist` RBAC permission are documented in `AI.md`. Env keys: `MEFOR_AI_MODE`, `MEFOR_AI_DATA_SCOPE`, `MEFOR_AI_ENVIRONMENT`, etc.

[logging]

Key	Type	Default	Notes
<code>level</code>	enum	<code>info</code>	log level; never run prod at <code>debug</code> (PHI) — <code>serve</code> refuses it (Gate #1)
<code>format</code>	enum	<code>text</code>	stdout rendering: <code>text</code> (default) or structured <code>json</code> (one object per line). Stdlib only — no structlog
<code>log_dir</code>	str	<code>unset</code>	Directory where NSSM or another supervisor stores and rotates captured stdout/stderr. The engine logs to stdout and does not write log files itself. <code>GET /status</code> reports this directory's total bytes and filesystem free space beside database metrics (#50). If unset, no directory measurement occurs. The engine reads metadata only, never file contents.
<code>forward_enabled</code>	bool	<code>derived</code>	Forward each log record to a syslog/SIEM collector, preserving off-host evidence after a host compromise (sec-offbox-log). If unset, forwarding is enabled only when <code>forward_host</code> is set (ADR 0080). Set <code>false</code> to disable forwarding even with a configured host. Without a host, logging remains stdout-only.
<code>forward_host</code>	str	—	syslog/SIEM collector host. Setting it turns forwarding on by default (above)
<code>forward_port</code>	int	<code>514</code>	collector port (1–65535)
<code>forward_protocol</code>	enum	<code>udp</code>	Supports <code>udp</code> , <code>tcp</code> , and <code>tls</code> (RFC 5425, native <code>ssl</code> -wrapped TCP, ADR 0080). UDP does not wait for confirmation. If a TCP or TLS collector is unavailable at startup, the engine warns and skips it. Socket timeouts limit runtime stalls and drop the affected record. The TLS handshake also has a time limit. Sending is synchronous, so prefer UDP or a local agent for high volume.

Key	Type	Default	Notes
<code>forward_format</code>	enum	<code>json</code>	Forwarded format, independent of stdout <code>format</code> . JSON writes one record per line. Text framing is best-effort because tracebacks can span multiple lines.
<code>forward_tls_ca_file</code>	str	—	PEM trust anchor for the collector certificate. Required when <code>forward_protocol = "tls"</code> and verification is enabled. Only this CA is trusted. The public system bundle is not loaded.
<code>forward_tls_verify</code>	bool	<code>true</code>	Verify the collector certificate and hostname. <code>false</code> selects the insecure exception: <code>CERT_NONE</code> , with no CA file required. Use that exception only for a lab or pinned network.
<code>forward_tls_client_cert</code>	str	—	optional PEM cert+key chain for mutual TLS to the collector
<code>forward_hop_attested</code>	bool	<code>false</code>	acknowledged opt-out for a plaintext / unverified-TLS collector hop (#200, ADR 0092 — the <code>[logging]</code> sibling of a connection's <code>tls_hop_attested</code>). A hop that is not verified TLS is now decided by the shared posture gradient: refused on an enforcing PHI instance, warned on a non-enforcing PHI instance, allowed for loopback / synthetic. Set this (with a reason) to affirm the hop is secure by other means — e.g. a dedicated out-of-band management VLAN
<code>forward_hop_attested_reason</code>	str	—	Reason that the collector connection is secure. Recorded for audit. Required and non-empty when <code>forward_hop_attested = true</code> (ADR 0153). The flag alone fails configuration load. A reason without the flag is also rejected.
<code>require_time_sync</code>	bool	<code>false</code>	opt-in startup clock-sync gate (ASVS 16.2.2, ADR 0080): before listeners start, probe <code>ntp_peer</code> and warn on skew. Requires <code>ntp_peer</code> . Default = no-op
<code>ntp_peer</code>	str	—	NTP/SNTP host to compare the local clock against (required when <code>require_time_sync</code>)
<code>time_sync_max_skew_seconds</code>	float	<code>2.0</code>	<code> local - peer </code> above this is "skewed" (must be > 0)
<code>time_sync_fail_closed</code>	bool	<code>false</code>	refuse to start (instead of warn) on skew or an unreachable peer. Further opt-in; requires <code>require_time_sync</code>
<code>file, max_bytes, backups</code>	str/int	—	accepted-but-ignored (planned) rotation — none is a <code>LoggingSettings</code> field. The engine logs to stdout and NSSM rotates it; <code>log_dir</code> above is how you point the engine at where it lands

Handler filters always redact PHI and remove control characters from every sink, including the external forwarder. These filters cannot be disabled. For encrypted forwarding, set `forward_protocol = "tls"` (native RFC 5425). Alternatively, put a local TLS-forwarding agent or trusted network in front of plaintext `udp` or `tcp` forwarding. See [PHI.md §7](#) and [ADR 0080](#).

The forwarded stream still includes usernames, connection names, message IDs, client addresses, and the tamper-evident audit chain. Before it installs the handler, `serve` applies the transport rules to the forwarding connection (#200, ADR 0092). Without verified TLS, an enforcing PHI instance refuses the connection. A non-enforcing PHI instance gives a warning. Loopback collectors and synthetic instances can use plaintext forwarding. Thus, plaintext to `127.0.0.1` through a local agent still works. For external forwarding, select `forward_protocol = "tls"` or declare `forward_hop_attested` with a reason.

[retention]

The engine's `retention task` removes expired PHI bodies. It sets the body and `messages.metadata` to NULL in the same statement (ASVS 14.2.7). It keeps the message row, counts, disposition, and audit trail, as in the Mirth Data-Pruner pattern. It never deletes a `messages` row or changes a body still in flight.

The raw `[retention]` fields default to `0` or `""`, which means keep or off. The `serve` command applies additional rules to PHI instances:

- With `[security].enforcement = enforce` (the default), unbounded `[security].delete_message_bodies_after_days` or `[retention].dead_letter_days` causes startup refusal (exit 2).
- Without enforcement, each unset PHI retention period becomes 30 days.
- The built-in environment names `dev`, `staging`, and `prod` all imply PHI.
- The audited opt-out is `[security].allow_keeping_phi_indefinitely = true`.

See PHI.md §8.

Key	Type	Default	Notes
<code>messages_days</code>			→ moved to <code>[security].delete_message_bodies_after_days</code> (ADR 0118) — set it there; no longer accepted in <code>[retention]</code> .
<code>dead_letter_days</code>	int	0	past N days, null the bodies of dead-lettered outbound rows (their own window — a dead row stays replayable until purged). 0 = keep
<code>allow_unbounded_phi</code>			→ moved to <code>[security].allow_keeping_phi_indefinitely</code> (ADR 0118) — set it there; no longer accepted in <code>[retention]</code> .
<code>state_max_age_days</code>	int	0	past N days, delete transform-state entries (ADR 0005) last written before the cutoff — keeps the in-memory state cache + table bounded. A simple global age purge (by <code>set_at</code>); per-namespace policy is a follow-up. 0 = keep
<code>connection_event_retention_hours</code>	int	0	past N hours , delete <code>connection_event</code> rows (the <code>[diagnostics]</code> #46 transport/lifecycle log — high-volume under a connect-per-message sender or a probe storm, so its own short window in hours , not days). 0 = inherit the <code>messages_days</code> body window (the ADR 0021 §7.5 default).
<code>app_log_days</code>	int	0	past N days, delete application log files (<code>.log / .txt</code> , one level) from the configured <code>[logging].log_dir</code> (#120). The supervisor (NSSM <code>AppRotateBytes</code>) rotates the daily logs by size but never by age , so the log dir grows unbounded; this bounds it (by file mtime, so the currently-written file is never eligible). 0 = keep. No-op unless <code>[logging].log_dir</code> is set . Metadata only — file content is never read. While <code>app_log_compress_days</code> is on, the same window also ages out the <code>*.log.gz / *.txt.gz</code> archives that setting produces — so compressing a log doesn't make it immortal; with compression off the eligible set is exactly what it was
<code>app_log_compress_days</code>	int	0	past N days, gzip application log files (<code>.log / .txt</code> , one level — the same selection as <code>app_log_days</code> , by mtime, so the currently-written file is never eligible) in <code>[logging].log_dir</code> to <code><name>.gz</code> (#119). The log stays readable (<code>gzip -d</code>) at a fraction of the disk, so a long-running box keeps far more history for the same footprint. Each file is free-space prechecked (<code>shutil.disk_usage</code> must show room for the source plus its archive plus a <code>max(10%, 1 MiB)</code> margin — short, and the file is skipped and logged , never attempted) and each written archive is integrity-validated — staged to an exclusively created, randomly named temp file beside it (<code>tempfile.mkstemp: O_CREAT O_EXCL</code> , so it never truncates an existing file, never follows a symlink, and never collides with a sibling engine shard compressing the same directory), <code>fsync</code> ed, re-read off disk , decompressed and compared byte-for-byte against the original, renamed into place, and then validated again at <code><name>.gz</code> itself — and it is that last check, on the bytes actually sitting where the log used to be, that authorizes removing the original. Any failure leaves the original in place , does not count it as compressed, and logs it; an existing <code><name>.gz</code> is never clobbered. The archive inherits the source's mtime, so <code>app_log_days</code> still ages it out. Files over 64 MiB are skipped (the codec is in-memory), and so is a file whose archive would not be smaller than it (an empty or already-compressed log — compressing must never cost disk). Names/counts/sizes are logged, never file content . 0 = never compress. No-op unless <code>[logging].log_dir</code> is set . Set it shorter than <code>app_log_days</code> — a longer window compresses nothing, since the delete sweep runs first
<code>search_preset_days</code>	int	0	past N days, delete saved-search presets (ADR 0136) neither used nor edited since the cutoff. The stored <code>criteria</code> is the operator's own content/ <code>field_value</code> needle — PHI-shaped by construction , encrypted at rest — so it needs a window like any other PHI tier (ASVS 14.2.7). The whole row is deleted, not blanked: a preset's entire payload <i>is</i> its criteria. Keys on last-USED (BACKLOG #306) — the cutoff is compared against the <i>later</i> of <code>updated_at</code> (written by a save) and <code>last_used_at</code> (written by a recall), so a preset you run daily but never re-save is kept . A preset last touched before the <code>last_used_at</code> column existed has it NULL and ages out on <code>updated_at</code> alone. 0 = keep forever (the default)
<code>audit_days</code>	int	0	reserved / not enforced . The <code>audit_log</code> is a tamper-evident hash chain and HIPAA expects ~6-year retention, so audit is keep-forever by design ; archive-first pruning is a tracked follow-up. Accepted so a forward-looking file still loads

Key	Type	Default	Notes
<code>max_db_mb</code>	int	<code>0</code>	advisory only: warn (WARNING log + an <code>AlertSink storage_threshold</code> event) when the database exceeds this — measured as the SQLite file + <code>-wal / -shm</code> , <code>SUM(size)</code> over <code>sys.database_files</code> on SQL Server , and <code>pg_database_size()</code> on Postgres . Never auto-deletes. <code>0</code> = off
<code>purge_interval_seconds</code>	float	<code>3600</code>	how often the purge/maintenance loop runs a pass
<code>max_pass_seconds</code>	float	<code>0</code>	maximum wall-clock seconds one maintenance pass may spend (#121, ADR 0137). A between-phase soft cap : <code>run_once</code> checks elapsed monotonic time before each phase and, once this is reached, skips the remaining phases (marking the pass <code>capped</code>) so a long pass can't run unbounded into the next maintenance window — the skipped tail re-runs next interval, and a skipped WAL-checkpoint/VACUUM does not advance its last-run marker. Checked only <i>between</i> phases, never inside one, so a running VACUUM is non-interruptible. <code>0</code> = off (the default — no cap); ~ <code>14400</code> (4 h, the Corepoint off-peak ceiling) is the recommended value when enabled
<code>wal_checkpoint_seconds</code>	float	<code>0</code>	<code>PRAGMA wal_checkpoint(TRUNCATE)</code> cadence — SQLite only; a documented no-op on SQL Server and Postgres , where log management is a DBA operation. <code>0</code> = off (rely on auto-checkpoint). Evaluated once per pass, so a value below <code>purge_interval_seconds</code> is effectively rounded up to it
<code>vacuum_at</code>	str	<code>""</code>	daily local <code>"HH:MM"</code> to run <code>VACUUM</code> (reclaims space freed by purges) — SQLite only; a documented no-op on SQL Server and Postgres , where space reclamation is a DBA operation. <code>""</code> = off. A daily off-peak time, not a cron expression (no new dependency); VACUUM holds a write lock on the whole DB while it runs

Per-connection overrides ([ADR 0027](#)). `messages_days` and `dead_letter_days` are **global defaults** an individual connection may override: an **inbound** sets its own `messages_days`, an **outbound** its own `dead_letter_days` (both `None` = inherit this global window, `0` = keep that connection's bodies forever, `>0` = days). An inbound may also opt into **embedded-document pruning** (`prune_documents_after` + `prune_documents_min_bytes`, [ADR 0042](#)) to strip bulky base64 attachments while keeping the readable message. These live on the connection (code-first or in `connections.toml`) — see [CONNECTIONS.md](#).

Backend coverage. The retention/purge pass is **backend-agnostic** and every PHI purge runs on **all three** backends (SQLite, SQL Server, Postgres) — `pipeline/retention.py` contains no backend branch. Only `wal_checkpoint_seconds` and `vacuum_at` are SQLite-only: on the server backends those methods are documented no-ops, and log management / space reclamation (plus the DB-tier `[backup]` snapshot) are DBA operations there. Each pass that does real work writes one `retention_purge audit_log` entry (cutoffs + counts, **no** message content). Per-backend table: [PHI.md §8](#).

[`update_check`]

The engine compares its version with local installed metadata ([ADR 0026](#)). It compares `messagefoundry.__version__` with `importlib.metadata` or the bundled `requirements.lock`. This comparison creates no outbound traffic.

The result adds a field to `/status` and optionally produces an `update_available` `AlertSink` event. The console and IDE show a dismissible banner. Neither calls PyPI. The local comparison is on by default.

Key	Type	Default	Notes
<code>enabled</code>	bool	<code>true</code>	emit the <code>/status</code> field + the <code>update_available</code> alert. <code>false</code> = suppress both
<code>check_interval_seconds</code>	float	<code>86400</code>	Interval for the version comparison. The daily default is sufficient for the small comparison. Must be <code>> 0</code> .
<code>mode</code>	str	<code>"local"</code>	the no-network diff (the only MVP value). <code>"live"</code> (the constrained-egress path, ADR 0026 §2) is defined but rejected at load , so a config can never silently turn the check into a phone-home out of a PHI system
<code>index_url</code> , <code>index_allowed_hosts</code>	str / list	unset	forward-compat for the future <code>"live"</code> mode only — accepted-but-unused in the MVP

[delivery]

Key	Type	Default	Notes
<code>retry_max_attempts</code>	int	<code>unset</code>	attempts before a delivery dead-letters. Unset = retry forever (the conservative default — a transient failure/ <code>AE</code> NAK is never silently lost; under FIFO the head blocks its lane until it succeeds or is purged). Set a finite value to opt back into retry-then-dead-letter. A permanent <code>AR</code> reject fails fast regardless.
<code>retry_backoff_seconds</code> , <code>retry_backoff_multiplier</code> , <code>retry_max_backoff_seconds</code>	num	5 / 2 / 300	exponential backoff between attempts (per-outbound <code>retry=</code> overrides)
<code>ordering</code>	enum	<code>fifo</code>	default queue ordering per outbound: <code>fifo</code> (strict in-order, head-of-line on failure) or <code>unordered</code> (batch + rotate-past-failures). Per-outbound <code>ordering=</code> overrides.
<code>internal_error</code>	enum	<code>continue</code>	what a delivery worker does on an internal/code error (a non- <code>DeliveryError</code> exception from <code>send</code> — our bug, not the partner's): <code>continue</code> (dead-letter the row + advance) or <code>stop</code> (halt the connection's worker, preserve the message for replay, raise a <code>connection_stopped</code> alert). Per-outbound <code>internal_error=</code> overrides. Partner NAKs / transport failures are unaffected.
<code>buildup_max_depth</code>	int	<code>unset</code>	raise a <code>queue_buildup</code> alert when an outbound lane's pending depth reaches this. Unset = depth dimension off (a healthy ceiling is throughput-specific, so there's no safe default). Per-outbound <code>buildup=BuildupThreshold(...)</code> overrides.
<code>buildup_max_oldest_seconds</code>	num	300	raise <code>queue_buildup</code> when the lane's oldest pending message has waited this long (a stuck/retry-forever head is the classic cause). On by default — a head stuck >5 min is a problem in any environment. Set to unset/ <code>0</code> -disable via a per-outbound override.
<code>stall_max_oldest_seconds</code>	num	<code>unset</code>	raise a <code>message_stall</code> alert (Corepoint "Max Message Stall", ADR 0014) when an outbound lane's oldest undelivered message has waited this long. Unset (the default) = the stall alert is OFF — deny-by-default/opt-in, because it overlaps <code>buildup_max_oldest_seconds</code> 's age dimension and would double-page if both fired. Set a threshold to turn it on; a per-outbound <code>stall=StallThreshold(...)</code> overrides it. The stall event routes through <code>[[alerts.rules]]</code> like any other (ADR 0014).
<code>saturation_sustain_samples</code>	int	<code>unset</code>	raise a <code>saturation</code> alert (BACKLOG #93, ADR 0014 amendment) when an outbound lane's pending backlog is rising sustained over this many consecutive samples — the queue derivative (ingest > drain), distinct from the absolute depth/age ceilings above. A bursty-but- draining lane (spike then fall) never fires; only a lane whose depth climbs monotonically does. Unset (the default) = OFF — deny-by-default/opt-in (it overlaps <code>buildup_max_oldest_seconds</code> 's age dimension). Floor of 2 (fewer can't tell a burst from sustained growth). Global-only for now; a per-outbound override is a documented follow-up (a <code>[[alerts.rules]]</code> <code>connection</code> glob with <code>transports = []</code> can suppress it for a known-bursty feed in the interim).
<code>priority</code>	enum	<code>normal</code>	global DR / priority tier default for every connection (#61, ADR 0048). A connection declaring no <code>priority=</code> of its own inherits this; resolution order is per-connection override > this global default > the built-in <code>normal</code> . The total order is <code>critical > normal > low</code> , and the <code>[dr]</code> run-profile starts only connections whose resolved rank is at or above <code>[dr].priority_threshold</code> . It governs when a connection runs , never what it does. <code>normal</code> keeps every connection at the same tier, so a deployment that never enables DR is byte-unchanged. An unknown value fails config load.
<code>outbox_workers</code>	int	—	accepted-but-ignored (planned): delivery concurrency. Not a <code>DeliverySettings</code> field — worker topology is set by <code>[pipeline].claim_mode</code> today
<code>dead_letter</code>	enum	—	accepted-but-ignored (planned): <code>keep / drop</code> -after-N. Not a <code>DeliverySettings</code> field — a finite <code>retry_max_attempts</code> is what dead-letters a row today

[pipeline]

Key	Type	Default	Notes
<code>max_correlation_depth</code>	int (≥ 1)	8	Re-ingress loop cap (ADR 0013 Increment 2). When a captured reply is re-ingressed (<code>reingress_to= / Loopback()</code>), the re-ingressed message carries a <code>correlation_depth</code> ; a message at this depth still routes, but the next hop (depth+1) dead-letters its re-ingress work-row and marks the origin <code>ERROR</code> . Coarse by design — it bounds <i>total work</i> , not topology, so a chain that legitimately bounces A→B→A a few times needs headroom. 8 is safe for typical request→response→route feeds; raise it for deep correlation chains, lower it to fence a misbehaving loop. (A value of 0 would dead-letter every re-ingress, so the floor is 1.)
<code>per_lane_wake</code>	bool	<code>false</code>	Per-lane wake events (B12, ADR 0061). Reliability-core, default-OFF . When <code>false</code> , a committed message wakes every worker of its stage via an engine-wide event (the historical behavior). When <code>true</code> , it wakes only its own (stage, lane) worker , eliminating the thundering-herd empty-claim storm that dominates at high connection counts (~1,500 inbounds). Correctness is unchanged (the FIFO claim + the 0.25 s lost-wakeup poll backstop are untouched; a missed wake self-heals within the poll). Read once at engine start — a <code>/config/reload</code> does NOT toggle it (restart to change) . Env override (for the connection-scale harness A/B): <code>MEFOR_PIPELINE_PER_LANE_WAKE=true</code> . Applies only in <code>per_lane</code> claim mode (see <code>claim_mode</code>); the default <code>pooled</code> mode routes wakes through its dispatchers instead, so this knob is inert there.
<code>claim_mode</code>	enum	<code>pooled</code>	Pipeline claim mode (ADR 0066). Reliability-core . <code>pooled</code> (the default since #744) runs one <code>StageDispatcher</code> per stage — a handful of shared claimer tasks batch-claim head-prefixes across lanes, collapsing the per-connection claim storm and holding zero-loss at high fan-out where <code>per_lane</code> drops messages. <code>per_lane</code> is the byte-identical opt-out (<code>[pipeline].claim_mode = "per_lane"</code>): the pre-ADR-0066 topology of one router+transform worker per inbound and one delivery worker per outbound, enforced by a test sentinel. Read once at engine start — a <code>/config/reload</code> does NOT toggle it (restart to change) . Env override (harness A/B): <code>MEFOR_PIPELINE_CLAIM_MODE</code> . Two caveats (see <code>CONNECTIONS.md</code> "Pipeline claim mode"): exactly-once degrades under load (no inbound de-dup — receivers must be idempotent; not pooled-specific) and active-passive failover-under-load is covered (the gated <code>test_load_failover_{postgres,sqlserver}</code> two-node kill-the-leader runs hold no-acknowledged-loss / per-lane FIFO / bounded dup-rate under pooled; only recovery <i>time</i> is host-dependent, and the T17 infra-fault spin is bounded by ADR 0070). Invariants (at-least-once / per-lane FIFO / poison-guard) are unchanged in both modes.
<code>pooled_claimers_per_stage</code>	int (≥ 1)	1	Pooled-only: K claimer tasks per stage (>1 hash-partitions lanes across claimers so no two claim the same lane).
<code>pooled_sweep_interval</code>	float (>0)	0.25	Pooled-only: the clock-driven discovery-sweep interval (the bounded at-least-once backstop; 0.25 s = <code>poll_interval</code> parity).
<code>pooled_claim_lane_chunk</code>	int (1–500)	256	Pooled-only: max lanes batch-claimed per claim round-trip (clamped down to the backend store's chunk — SQLite 200, SS/PG 500).
<code>pooled_max_processing_lanes</code>	int (≥ 1)	256	Pooled-only: max concurrently-processing lanes per stage (the decrypted-body / crash-exposure bound).
<code>require_rcsi_for_pooled</code>	bool	<code>true</code>	Pooled-only (SQL Server): fail closed at startup if <code>READ_COMMITTED_SNAPSHOT</code> is OFF (the §3.2 correctness proofs assume RCSI on); <code>false</code> downgrades to a loud warning + a <code>/stats rcsi_off_degraded</code> gauge. No-op on SQLite/Postgres.
<code>infra_fault_policy</code>	enum	<code>stop</code>	Pooled T17 (infra/machinery-fault) handling (ADR 0070). A store/handoff error — or any raise from outside the per-item body — is caught by the dispatcher's T17 handler, which always re-pends the faulting head at an exponential-capped backoff (collapsing the ~4×/s sweep spin). This key bounds a persistent such fault: <code>stop</code> (default) STOPS the head-of-line-blocked lane after <code>infra_fault_stop_after</code> consecutive zero-progress faults, reusing the <code>internal_error = stop</code> muscle (STOPPED phase + <code>connection_stopped</code> alert; reload / new work re-arms) and never dead-lettering the good message. <code>retry_forever</code> never STOPS — it retries the head at capped backoff forever and emits a throttled <code>lane_stuck</code> alert once the horizon is crossed (for a deliberately-unattended flaky-infra site). Reliability-core: read once at engine construction — a <code>/config/reload</code> does NOT re-read it (restart to change) .

Key	Type	Default	Notes
<code>infra_fault_stop_after</code>	int (≥1)	10	consecutive zero-progress T17 faults before a <code>stop</code> -policy lane transitions to STOPPED — and the stuck horizon at which <code>retry_forever</code> 's throttled <code>lane_stuck</code> alert first fires. Under the exponential backoff (capped by <code>infra_fault_backoff_cap</code>) 10 spans ~4 min of wall clock, so it is really a duration gate.
<code>infra_fault_backoff_cap</code>	float (>0)	60.0	cap (seconds) on the T17 head re-pend backoff — base is the dispatcher's 1 s lane-error backoff, doubling per consecutive zero-progress fault. ~60 s picks a recovered dependency back up within about a minute while still collapsing the spin.
<code>fuse_thread_hops</code>	bool	<code>false</code>	Thread-hop fusion (ADR 0071 B5). Reliability-core, default-OFF, SQL-Server-scoped : when <code>true</code> and the store backend is SQL Server and <code>claim_mode</code> = "pooled", each fused stage (INGRESS/ROUTED) runs its off-loop CPU stage (<code>route_only</code> / <code>transform_one</code>) together with its store handoff on a single dedicated-executor worker hop, collapsing a multi-statement aiiodbc handoff into one executor→loop completion (the profiled per-completion async-marshaling wall). Provably no-op elsewhere: Postgres (asynpg is loop-native) and SQLite (loop-affine handoff lock) keep the async path by construction and log "ignored", and a sync-handoff-pool open failure downgrades to the async path with a loud warning + a degraded gauge — never a lane outage. Read once at engine construction (restart to change) . Env/harness A/B: <code>MEFOR_PIPELINE_FUSE_THREAD_HOPS</code> .
<code>pooled_fusing_workers</code>	int (≥1)	8	worker count for each per-stage fusing executor (ADR 0071 B5). Every fused stage gets its own <code>ThreadPoolExecutor</code> of this width plus a matching-width dedicated synchronous pyodbc handoff pool (one connection per worker, so a fused hop never blocks acquiring). Small by default — a fused hop holds a worker across DB latency, so this is the fused-stage concurrency; it also clamps the fused stages' effective <code>pooled_max_processing_lanes</code> to ~2× this value, so the claimer doesn't reserve 256 slots for a handful of workers. Inert unless <code>fuse_thread_hops</code> is on.
<code>batch_handoff_statements</code>	bool	<code>true</code>	Per-hop SQL statement batching (ADR 0075). Default-ON (promoted 2026-07-08; retained as an emergency off-switch) and SQL-Server-scoped: each per-hop staged handoff (<code>route_handoff</code> / <code>transform_handoff</code>) folds the non-result-returning DML of its body into the fewest <code>pyodbc.execute()</code> T-SQL batches — the same ordered (<code>sql</code> , <code>params</code>) sequence, one round-trip per batch, still committing exactly once per hop . It cuts network round-trips, not transactions: no commit boundary moves, the claim stays its own poison-guard transaction, and the ACK-on-receipt fence is untouched. Each result-consuming statement whose value gates later control flow (the guard <code>DELETE</code> , the finalize <code>GROUP BY</code> , the finalize <code>sp_getapplock</code> rc-check) stays its own execute. Postgres and SQLite have no batched path and run byte-identically. Read once at engine construction (restart to change) . Env: <code>MEFOR_PIPELINE_BATCH_HANDOFF_STATEMENTS</code> .
<code>snapshot_on_send</code>	bool	<code>true</code>	Copy-on- Send (ADR 0104) — snapshot each <code>Send</code> 's payload at construction so a divergent fan-out (mutate between <code>Send</code> s) delivers per-destination state instead of a last-write collapse. Default-ON since the BACKLOG #230 default-flip: the conservative estate AST scan flagged 1 of 152 handlers and human triage found that one mutates an independent clone, so genuine divergence is 0 and the flip changes delivered bytes for no handler; and <code>Message.copy()</code> is now genuine copy-on-write, so the common single- <code>Send</code> / no-post-mutation path is zero-copy and a deepcopy fires only on an actual divergence. Set <code>false</code> to restore the pre-ADR-0104 last-write behaviour. Backend-agnostic. Read once at engine construction (restart to change) . Env: <code>MEFOR_PIPELINE_SNAPSHOT_ON_SEND</code> .
<code>credential_fault_policy</code>	enum	<code>stop</code>	Partner-account-lockout protection (#109, ADR 0095). What an outbound File/FTP/SFTP sender does on a permanent credential/auth fault (bad password, key rejected). <code>stop</code> (default) halts the lane immediately (not after a streak) and retains the queued rows un-errored (they stay pending/claimable, never dead-lettered), so a backlog can't re-authenticate in a loop and trip the partner's account lockout — reusing the STOP muscle (<code>connection_stopped</code> alert; reload/restart re-arms the lane once the credential is fixed). <code>dead_letter</code> keeps the historical fail-fast (dead-letter just the offending row and advance). A content -permanent reject (AR/CR, no-such-dir) is unaffected — it still dead-letters.
<code>schedule_tick_seconds</code>	float (>0)	30.0	Active-window scheduler tick (#147, ADR 0095). The reconcile granularity for a connection's per-connection <code>schedule</code> (a window boundary is honoured within one tick). Only affects connections that declare a <code>schedule</code> ; connections with none are byte-identical always-on.

[sandbox]

This section provides optional subprocess isolation for Routers and Handlers. See [ADR 0087](#), [BACKLOG #197](#), and [ASVS 15.2.5](#). Administrators write Routers and Handlers in Python. By default, these functions share the engine address space with the DEK, audit chain, and live sockets.

The default, `mode="off"`, leaves in-process execution unchanged with no added overhead. With `mode="subprocess"`, each inbound has a persistent child worker. The engine does not create a child for each message. The worker blocks socket, store, and cryptography imports and enforces the resource limits below. It refuses the live `db_lookup` and `fhir_lookup` bridges because they need access to the engine event loop. For Handlers that need live enrichment, use `mode=off`.

A forbidden operation, exceeded limit, or worker crash causes `ERROR` or dead-letter handling after acknowledgement. The engine does not send a NAK or drop the message. The engine reads these settings only at startup. Restart it to apply changes. A `/config/reload` does not reread them.

Key	Type	Default	Notes
<code>mode</code>	enum	<code>off</code>	<code>off</code> (in-process, byte-identical, no subprocess) or <code>subprocess</code> (persistent per-inbound worker child).
<code>wall_seconds</code>	float (>0)	5.0	Authoritative wall-clock cap per Router/Handler call on every platform — the parent kills a worker that overruns it, so a pathological busy-loop can't wedge intake.
<code>cpu_seconds</code>	float (>0)	2.0	POSIX-only <code>RLIMIT_CPU</code> backstop inside the child (a no-op on Windows, where <code>wall_seconds</code> governs).
<code>mem_mb</code>	int (≥ 1) or null	512	POSIX-only <code>RLIMIT_AS</code> address-space cap (MiB) inside the child (no-op on Windows). <code>null</code> disables it.
<code>startup_seconds</code>	float (>0)	30.0	Bound on the one-time child bootstrap (config load + guard install) before start fails closed.

[diagnostics]

The event log records connection lifecycle events, failures before ingress, and returned ACK/NAK dispositions (#46). Both master switches are on by default. The log stores non-PHI metadata: connection name, peer IP, scrubbed reason, and ACK disposition. It stores an AA-ACK body only with an encrypted store. Otherwise, that body is NULL. It never stores a NAK body.

A connection's `capture_connection_errors` or `capture_ack` flag overrides the matching master switch. See [CONNECTIONS.md](#). The `message_events` setting controls per-message log detail. It defaults to `all` and retains the compliance minimum even at `off`. The fourth row describes a relocated key. The loader rejects it here ([ADR 0118](#)).

Key	Type	Default	Notes
<code>connection_events</code>	bool	<code>true</code>	master switch for the connection/transport event log : inbound lifecycle (established/closed) + pre-ingress failures (allowlist/capacity/oversize/peer-reset/framing) + outbound lane transitions (connection_lost/restored). Metadata-only, written off the hot path by a drain task. Per-connection <code>capture_connection_errors</code> overrides it.
<code>response_sent</code>	bool	<code>true</code>	master switch for "Response Sent" — the ACK/NAK returned to an inbound sender. Always captures the disposition metadata (<code>ack_code</code> / <code>phase</code> / <code>outcome</code>); the AA body is stored only on an encrypted store, and every NAK body is NULL. Per-connection <code>capture_ack</code> overrides it.
<code>message_events</code>	enum	<code>all</code>	verbosity of the per-message <code>message_events</code> disposition log (#63) — how many rows the store writes to that table. <code>all</code> (default) records every event; <code>errors</code> drops the routine successes (<code>received</code> / <code>delivered</code> / <code>replayed</code>); <code>off</code> keeps only the floor. A compliance floor is retained at every level, even <code>off</code> : <code>viewed</code> (the HIPAA PHI-access trail) plus the terminal <code>dead</code> / <code>error</code> / <code>failed</code> . Not a master switch — it never touches the <code>messages</code> /queue disposition rows (count-and-log is separate) or the tamper-evident <code>audit_log</code> chain.
<code>audit_all_authz</code>			→ moved to <code>[security].audit_all_authorization_decisions</code> (ADR 0118) — set it there; no longer accepted in <code>[diagnostics]</code> .

Retention for the event log has its own short window — `[retention].connection_event_retention_hours` (in **hours**; `0` = inherit `messages_days`).

[egress]

The outbound allowlists restrict where the engine can send PHI (WP-11c, ASVS 13.2.4/13.2.5/14.2.3). An empty list does not restrict that transport. With a list set, the engine refuses destinations outside it during configuration load or reload. It compares the resolved destination after `env()` substitution. Refusal raises a `WiringError`, causing HTTP 422 or a refused reload. The PHI startup rules below also apply.

Key	Type	Default	Notes
<code>allowed_mllp</code>	list	<code>[]</code>	allowed MLLP destinations; each entry is <code>host</code> (any port) or <code>host:port</code> . Via env: comma-separated <code>MEFOR_EGRESS_ALLOWED_MLLP</code>
<code>allowed_tcp</code>	list	<code>[]</code>	allowed raw-TCP (<code>Tcp(...)</code>) destinations; each entry is <code>host</code> (any port) or <code>host:port</code> . An inbound <code>Tcp(...)</code> is a local listener and is not gated. Via env: comma-separated <code>MEFOR_EGRESS_ALLOWED_TCP</code>
<code>allowed_file_dirs</code>	list	<code>[]</code>	allowed File output directories; a destination's directory must resolve at/under one of these
<code>allowed_http</code>	list	<code>[]</code>	allowed REST/SOAP (HTTP) destination hosts; each entry is <code>host</code> (any port) or <code>host:port</code> (ADR 0003). Via env: comma-separated <code>MEFOR_EGRESS_ALLOWED_HTTP</code>
<code>allowed_db</code>	list	<code>[]</code>	allowed DATABASE destination servers; each entry is <code>host</code> (any port) or <code>host:port</code> (ADR 0003). Via env: comma-separated <code>MEFOR_EGRESS_ALLOWED_DB</code>
<code>allowed_remote</code>	list	<code>[]</code>	allowed RemoteFile (SFTP/FTP/FTPS) hosts — gates the connector in both directions (source poll + destination upload); each entry is <code>host</code> (any port) or <code>host:port</code> . Via env: comma-separated <code>MEFOR_EGRESS_ALLOWED_REMOTE</code>
<code>allowed_smtp</code>	list	<code>[]</code>	allowed email (SMTP) destination hosts for the <code>Email(...)</code> outbound (ADR 0029); each entry is <code>host</code> (any port) or <code>host:port</code> . Distinct from <code>[alerts].smtp_allowed_hosts</code> , which gates the alert notifier's own SMTP dial. Via env: comma-separated <code>MEFOR_EGRESS_ALLOWED_SMTP</code>
<code>allowed_direct</code>	list	<code>[]</code>	allowed Direct (S/MIME-over-SMTP HISP relay) destination hosts (ADR 0085); each entry is <code>host</code> (any port) or <code>host:port</code> . Kept deliberately separate from <code>allowed_smtp</code> so an operator can permit a Direct HISP relay without opening generic email egress — a distinct trust relationship carrying encrypted PHI. Via env: comma-separated <code>MEFOR_EGRESS_ALLOWED_DIRECT</code>
<code>proxy_url</code>	str	<code>unset</code>	site-wide default forward/egress web proxy for the HTTP family — REST/SOAP/FHIR/ <code>fhir_lookup</code> /DICOMweb plus the OAuth2/SMART token endpoints (ADR 0126). A connection that sets no per-connection <code>proxy</code> inherits this; a per-connection value overrides it. Unset (default) = no site-wide proxy (byte-identical — only per-connection proxies apply). <code>"default"</code> selects the OS default web proxy (<code>getproxies()</code>); an <code>http(s)://</code> address names an explicit one. Proxy credentials stay per-connection (secrets via <code>env()</code>), never a global TOML value. Via env: <code>MEFOR_EGRESS_PROXY_URL</code>
<code>proxy_no_proxy</code>	list	<code>[]</code>	the site-wide <code>NO_PROXY</code> -style bypass list inherited by a connection that sets no per-connection <code>proxy_no_proxy</code> . Each entry is a host, <code>.suffix</code> , <code>*.suffix</code> or <code>*</code> . Via env: comma-separated <code>MEFOR_EGRESS_PROXY_NO_PROXY</code>
<code>deny_by_default</code>			→ moved to <code>[security].block_unlisted_outbound</code> (ADR 0118) — set it there; no longer accepted in <code>[egress]</code> .
<code>fhir_require_structured_params</code>	bool	<code>false</code>	when <code>true</code> , a <code>fhir_lookup</code> search must use the structured <code>params=</code> form (each value percent-encoded); the flat author-encoded <code>?-query</code> is refused before it dials out (ASVS 1.2.2, ADR 0043). A read-by-id and the <code>params=</code> form are unaffected. Default <code>false</code> keeps the flat form (byte-identical). Via env: <code>MEFOR_EGRESS_FHIR_REQUIRE_STRUCTURED_PARAMS</code>

FHIR query encoding (ASVS 1.2.2). For a Pass posture, set `[egress].fhir_require_structured_params = true`. This requires every `fhir_lookup` search to use `params=`, which encodes each value separately. It disables flat `?` queries that could introduce extra search parameters through incorrectly encoded values. The default is off for compatibility. In that mode, the Handler author must encode flat-query values correctly.

Unrestricted PHI egress causes startup refusal. This occurs when no `[egress].allowed_*` list is set and deny-by-default is off. With `[security].enforcement = enforce` (the default), `serve` exits 2 for any PHI instance. The names `dev`, `staging`, and `prod` all imply PHI. With `enforcement = warn`, the engine writes a warning to stderr instead. Synthetic instances are exempt. Restrict egress with transport allowlists, `[security].block_unlisted_outbound = true`, or both. The loader rejects the old `[egress].deny_by_default` key.

Webhook and SMTP alerts contain no message bodies or PHI. Their host allowlists are `[alerts].webhook_allowed_hosts` and `[alerts].smtp_allowed_hosts`.

Cleartext HTTP restrictions (ASVS 12.2.1). These rules apply separately from destination allowlists. See [ADR 0153](#). The `insecure_hop_disposition` function in `config/tls_policy.py` decides whether a cleartext connection is permitted. The `refuse_cleartext_egress` function enforces the result during construction (`transports/rest.py`). The rules apply in this order: 1. Permit loopback or on-box connections. 2. Permit a connection with `tls_hop_attested`. The operator declares protection by other means. 3. Permit a connection with `cleartext_accepted`, with a warning and audit record. The operator accepts the lack of protection. 4. Give a warning for `[security].enforcement = warn`. 5. Refuse all other connections.

These rules do not use `data_class`. A `synthetic` label no longer permits cleartext transport without a declaration. The rules also ignore `MEFOR_ALLOW_INSECURE_TLS`. That variable remains available for non-connection settings that cannot carry a declaration.

[shadow]

A shadow instance processes copied traffic for comparison with a legacy engine (#15). The legacy engine remains the live sender. The shadow must not deliver to live partners.

In simulate mode, an outbound runs the full pipeline, counts and logs the message, and completes it as `PROCESSED`. It sends no bytes or SQL to the partner. It retains the proposed payload for comparison.

Key	Type	Default	Notes
<code>simulate_all_egress</code>	bool	false	deployment-wide master switch: when <code>true</code> , every outbound runs egress-suppressed regardless of its own <code>simulate=</code> — so a shadow stand-up can't accidentally leave one outbound live. Default <code>false</code> = each outbound's own <code>simulate=</code> flag applies.

Per-outbound control is the precise mechanism — set `simulate = true` on an individual outbound (`outbound(..., simulate=True)` or `simulate = true` in `connections.toml`); this section is the blunt instance-wide override. A simulated lane is surfaced as `simulated` on `GET /connections` and shown as `[SIMULATED]` in the console. Simulate suppresses **egress only** — the `[egress]` allowlist, connector construction, and handler state writes are unaffected. With egress suppressed there is no real partner reply, so a **capturing / reingress_to** outbound captures (and re-ingresses) **nothing** in simulate — the message just finalizes `PROCESSED`.

Simulate is not "not deployed." A simulated outbound is **fully wired** — its connector is built, its `env()` values are resolved, it receives rows, and it finalizes `PROCESSED`; it just suppresses the bytes on the wire. A **not-deployed** connection (`deployed=false`, [ADR 0111](#)) is the opposite: it is never built, its `env()` is never resolved, and a `Send` to it is recorded-and-dropped, not delivered-to-nothing. Use `simulate` for parallel-run; use `not deployed` for a feed that exists in config but is deliberately dark. See [CONNECTIONS.md](#) → [Connection lifecycle](#).

[alerts]

This section sets destinations for operational alerts from the delivery pipeline. Events include `connection_stopped`, `queue_buildup`, `connection_error`, `message_stall`, and `integrity_drift`. Both transports are off by default. Without a configured transport, `LoggingAlertSink` logs events at `WARNING`. A transport starts when its required settings are present.

Payloads contain connection names and queue information, never message bodies or PHI. A background task attempts delivery without blocking a delivery lane. Delivery is best-effort.

Key	Type	Default	Notes
<code>webhook_url</code>	str	<code>unset</code>	enable the webhook transport: HTTP <code>POST</code> the event as JSON here (fronts Slack/Teams/PagerDuty/custom inbound webhooks).
<code>webhook_timeout</code>	num	10	seconds per POST
<code>webhook_allowed_hosts</code>	list	<code>[]</code>	egress allowlist for the webhook host (<code>[]</code> = any); SSRF defense (ASVS 15.3.2/1.3.6)
<code>email_smtp_host</code>	str	<code>unset</code>	SMTP server; with <code>email_from</code> + <code>email_to</code> set, enables the email transport
<code>email_smtp_port</code>	int	587	SMTP port
<code>email_from</code>	str	<code>unset</code>	sender address (required for email)
<code>email_to</code>	list	<code>unset</code>	recipient(s) (required for email). Via env: comma-separated <code>MEFOR_ALERTS_EMAIL_TO</code>
<code>email_use_tls</code>	bool	<code>true</code>	issue STARTTLS before sending
<code>email_username</code>	str	<code>unset</code>	SMTP login user (omit for unauthenticated relays)
<code>email_password</code>	str	<code>unset</code>	secret — supply via <code>MEFOR_ALERTS_EMAIL_PASSWORD</code> , never the file (or use <code>email_password_secret</code>)
<code>email_password_secret</code>	str	<code>unset</code>	connector <code>SecretProvider</code> reference (ADR 0019 §5) — when set and <code>[secrets].provider</code> is configured, the SMTP password is resolved from that backend (e.g. a Vault KV <code>path#field</code>) instead of <code>email_password</code> . A reference, not a secret.
<code>email_timeout</code>	num	30	seconds per send
<code>smtp_allowed_hosts</code>	list	<code>[]</code>	egress allowlist for the SMTP host (<code>[]</code> = any); parity with <code>webhook_allowed_hosts</code> (WP-11c)
<code>email_subject_template</code>	str	<code>unset</code>	optional operator-editable alert-email subject (#138, ADR 0127). Unset (the default, with its two siblings) = the fixed subject + key/value body, byte-identical to before. When set it is a <code>{name}</code> template over a closed non-PHI variable allow-list , validated at config load and fail-closed — an unknown or message-derived reference raises rather than rendering
<code>email_body_template</code>	str	<code>unset</code>	the same, for the plain-text body. The plain-text part is always sent, even when an HTML alternative is configured
<code>email_html_template</code>	str	<code>unset</code>	the same, adding an HTML alternative part whose substituted <i>values</i> are HTML-escaped. Never HTML-only — it supplements <code>email_body_template</code> , it does not replace it
<code>security_notifications_required</code>	bool	<code>true</code>	secure-by-default gate (BACKLOG #188, ASVS 6.3.5/6.3.7) . On a PHI instance, if no effective out-of-band security-notification channel is configured — <code>[auth].notify_security_events</code> on and <code>email_smtp_host</code> + <code>email_from</code> set — <code>serve</code> refuses to start (exit 2) . The refuse/warn split is <code>[security].enforcement</code> , not the production tier — the gate reads <code>enforcing</code> , <code>enforce</code> is the shipped default, and all three built-in env names derive PHI, so <code>serve --env staging</code> on stock defaults with no <code>[alerts]</code> SMTP is refused, not warned. It warns only under <code>enforcement = warn</code> . Set <code>false</code> to accept the pull-only <code>GET /me/security-events</code> feed instead (audited).
<code>realert_seconds</code>	num	300	suppress re-notifying the same (event, connection) more often than this (anti-spam for a flapping lane). A matching rule's <code>cooldown_seconds</code> overrides it.
<code>rules</code>	list	<code>[]</code>	ordered <code>[[alerts.rules]]</code> table array — per-event severity, transport routing, thresholds, suppression, cooldown (see below). Empty = today's behaviour (every event → every transport at <code>warning</code>).

`[[alerts.rules]]` — per-event routing (ADR 0014)

The `[[alerts.rules]]` array is ordered. The first matching rule wins. Put the most specific rules first. If no rule matches, the engine notifies every configured transport at `warning` with the global `realert_seconds`. Thus, a new rule does not suppress unnamed events. Rules use configuration values, not code or `eval`.

Key	Type	Default	Notes
event_type	str	any	match this event. The validator accepts <code>any</code> plus exactly these 18 , and rejects anything else at config load , so a typo is loud rather than a rule that silently never matches: <code>backup_failed</code> , <code>bootstrap_admin_expiring</code> , <code>cert_expiry</code> , <code>connection_error</code> , <code>connection_stopped</code> , <code>content_match</code> , <code>dr_activated</code> , <code>gcm_invocations</code> , <code>integrity_drift</code> , <code>lane_stuck</code> , <code>leadership_acquired</code> , <code>message_stall</code> , <code>queue_buildup</code> , <code>rcsi_off_degraded</code> , <code>saturation</code> , <code>secret_rotation</code> , <code>storage_threshold</code> , <code>update_available</code>
connection	str (glob)	*	glob over the connection name (e.g. <code>OB_*</code> , <code>IB_ACME_*</code>)
min_depth	int	unset	<code>queue_buildup</code> only — match only when pending depth is at/over this
min_oldest_seconds	num	unset	<code>queue_buildup</code> only — ...or the oldest pending message has waited at least this long
severity	str	warning	<code>info</code> <code>warning</code> <code>critical</code> — tagged onto the event (webhook JSON + email subject) for downstream triage
transports	list	all	which transports fire: subset of <code>["webhook", "email"]</code> ; unset = all configured ; <code>[]</code> = SUPPRESS (drop silently)
cooldown_seconds	num	global	override <code>realert_seconds</code> for matching events (e.g. re-page a critical sooner)

```
[alerts]
webhook_url = "https://hooks.example.com/services/XXX" # webhook transport on
email_smtp_host = "smtp.example.com" # email transport on
email_from = "alerts@example.com"
email_to = ["oncall@example.com"]

# Page (webhook) immediately and re-page every minute when any inbound connection stops.
[[alerts.rules]]
event_type = "connection_stopped"
connection = "IB_*"
severity = "critical"
transports = ["webhook"]
cooldown_seconds = 60

# A deep backlog on any outbound is critical; a shallow one only emails.
[[alerts.rules]]
event_type = "queue_buildup"
min_depth = 1000
severity = "critical"

[[alerts.rules]]
event_type = "queue_buildup"
severity = "info"
transports = ["email"]

# Stay quiet about a known-bursty test feed.
[[alerts.rules]]
connection = "OB_LOADTEST"
transports = [] # suppress every event for this connection
```

A rule routing to a transport that isn't configured (e.g. `transports = ["email"]` with no SMTP settings) is rejected at startup, so a typo can't silently black-hole an alert. Severity travels in the payload; **timed multi-stage escalation** ("email now, page in 15 min") is future work (ADR 0014 §3) — rules give the static routing primitive it would build on.

[cert_monitor]

The engine monitors TLS certificate expiry. It reads the public certificate's `notAfter` value, never the private key. It monitors `[api].tls_cert_file` and each connection's `tls_cert_file` for MLLP server and client identity. An expired certificate, or one within `warn_days` of expiry, causes a `cert_expiry` alert. Use a `[[alerts.rules]]` rule to route this alert. See `DEPLOYMENT.md` for native TLS outside loopback.

The monitor is on by default with a 30-day warning period. Set `warn_days = 0` to disable it.

Inbound mTLS caller certificates (ASVS 6.4.5). The engine verifies caller certificates but does not serve them. The cert-identity resolver examines each verified, allow-listed client certificate during the mTLS handshake. It raises the same alert within `warn_days` of expiry. The `check_interval_seconds` setting limits repeat alerts for each certificate.

List copies of caller certificates in `[api].tls_client_cert_files` to include them in file scans. This also monitors callers that stop connecting. A handshake can detect only a certificate that the caller presents.

Key	Type	Default	Notes
warn_days	int	30	alert when a served cert expires within this many days; <code>0</code> disables the monitor
check_interval_seconds	num	43200	rescan cadence (default 12h); the per-cert re-alert throttle is <code>[alerts].realert_seconds</code>

```
[cert_monitor]
warn_days = 45           # start warning 45 days out

# Page (don't just email) when a served cert is close to expiry.
[[alerts.rules]]
event_type = "cert_expiry"
severity = "critical"
transports = ["webhook"]
```

[secrets] — connector SecretProvider selection

Selects **how a named connector credential is sourced** (ADR 0019 §5) — from an external secrets backend **instead of** a `MEFOR_*` env var. The connector-secret twin of `[store].key_provider` (which sources the store DEK).

Key	Type	Default	Meaning
provider	str	none	<code>none</code> <code>env</code> <code>vault</code> . none (default) consults no provider — every credential stays env-sourced (byte-identical). <code>env</code> resolves a reference as an env-var name; <code>vault</code> reads Vault KV v2 behind the lazy <code>[vault]</code> / <code>hvac</code> extra (the same dependency the store's Vault <code>key_provider</code> uses — no new dep). Names a <i>provider</i> , not a secret.

A provider is consulted **only** for a credential whose per-credential `*_secret` reference is set — today `[auth].ad_bind_password_secret` and `[alerts].email_password_secret` (the wired points); the SQL Server store password is seam-only (managed identity is preferred there). A reference is `"<kv-path>"` or `"<kv-path>#<field>"` for `vault` (field defaults to `value`; KV mount from `MEFOR_SECRETS_VAULT_KV_MOUNT`, default `secret`); Vault address/token come from `MEFOR_SECRETS_VAULT_ADDR` / `MEFOR_SECRETS_VAULT_TOKEN` (falling back to `hvac's VAULT_ADDR` / `VAULT_TOKEN`). **Fail-closed:** a reference with `provider = none`, an unknown provider, a missing `[vault]` extra, or an unresolvable/empty secret raises at load/connect — never a blank credential; the value is never logged.

[secret_rotation]

The engine monitors secret age and raises rotation reminders (ADR 0019 §5.1). It compares each tracked secret's last rotation date with its maximum age. It raises an alert when rotation is overdue or within `warn_days` of its due date. The monitor reads rotation dates, never secret values.

Use `event_type = "secret_rotation"` in an alert rule. The internal `AlertSink` method name, `secret_rotation_due`, is not a valid rule value. The loader rejects it.

The reminder does not rotate keys or block startup. Use `rotate-key` to rotate the store data-encryption key (DEK). Under `[security].enforcement = enforce`, an overdue DEK produces an escalated alert at restart after `store_key_max_age_days + enforce_grace_days`. This sets `enforced = true` in the alert. It does not refuse startup.

The engine tracks the store DEK by default (ASVS 13.3.4). At the first keyed start, it records the key ID and first-seen date in store metadata. These values contain no secret. The `store_key_last_rotated` setting overrides this date but is not required.

The engine also tracks its connector, AD, SMTP, Vault, and OIDC credentials. It uses a DEK-derived keyed MAC to fingerprint each credential. A changed fingerprint resets the rotation date. The default warning period is 14 days for tracked secrets. Set `warn_days = 0` to disable the reminder.

Key	Type	Default	Notes
warn_days	int	14	alert when a tracked secret is due within this many days; 0 disables the reminder
check_interval_seconds	num	86400	rescan cadence (default 24h); the per-secret re-alert throttle is <code>[alerts].real_alert_seconds</code>
store_key_last_rotated	str	—	ISO <code>YYYY-MM-DD</code> the store DEK was last rotated; unset ⇒ the DEK is still tracked live-by-default off a persisted first-seen stamp (this date is an override)
store_key_max_age_days	int	365	rotate the store DEK within this many days of its effective last-rotated (the operator date if set, else the persisted stamp)
secret_max_age_days	int	365	max age for the non-DEK tracked secret classes (connector/AD/SMTP/Vault/OIDC), alerted this many days after their last observed fingerprint change
enforce_grace_days	int	30	under <code>[security].enforcement=enforce</code> , a DEK older than <code>store_key_max_age_days + this</code> escalates its rotation alert at restart (still an alert, never a refusal)

```
[secret_rotation]
store_key_last_rotated = "2026-01-15" # when you last ran rotate-key
store_key_max_age_days = 365          # remind me a year later
warn_days = 30                       # start 30 days ahead

# Page when the store DEK is overdue for rotation.
[[alerts.rules]]
event_type = "secret_rotation"
severity = "warning"
transports = ["email"]
```

[cluster] — active-passive HA coordination (Track B)

Clustering requires a server database. It provides node membership, heartbeats, leader election, and cross-node reference and configuration convergence. The supported model is **active-passive high availability**: one leader runs the graph, and a standby takes over after failure. The active-active design was dropped on 2026-06-18. Its code was removed, and it is not planned.

With `enabled = false` (the default), the no-op coordinator leaves single-node behavior unchanged. Clustering requires `[store].pool_size >= 2`. The loader rejects a smaller pool or a store without clustering support. Use a pool of at least 3 where possible. Cluster maintenance and stage workers share the pool. A pool of 1 would serialize their operations.

The supported backends are:

- **Postgres**: leader election, row leases, and a leader-only sweep for expired leases.
- **SQL Server**: the same self-fencing leadership lease. Only the leader processes messages.

Both backends recover in-flight rows on promotion. The periodic `reclaim_expired_leases` sweep applies to Postgres. SQLite does not support clustering.

On Postgres, the leader holds the `leader_lease` row (Workstream A2). It renews the lease every `heartbeat_seconds` to `DB_now + leader_lease_ttl_seconds`. The database clock determines lease ownership, so node clock differences do not affect this decision. A standby can acquire leadership only after the lease expires.

A leader that cannot renew within `leader_fence_timeout_seconds` stops its leader operations. This timeout must be less than `leader_lease_ttl_seconds`. Thus, the old leader stops before a standby can acquire the expired lease.

Only the leader runs these write operations:

- The `[retention]` purge, VACUUM, and audit tasks.
- The expired-lease sweep. It calls `reclaim_expired_leases` every `reclaim_interval_seconds` to recover rows from crashed nodes.

The sweep recovers only expired leases. It does not take work from a live node. Clustered startup skips the single-node `reset_stale_inflight` operation because that operation would take live sibling rows. Followers do not run these write tasks. After failover, the new leader starts them on the next poll.

Only the leader polls sources (Track B Step 4b). File, database, and remote-file sources read shared external resources. Remote-file sources include SFTP and FTP directories. Multiple nodes that poll the same resource could accept the same file or row twice.

The leader also runs listen sources (`mllp` , `tcp`) and all router, transform, and delivery workers. A standby binds no listeners and runs no workers. The queue's `FOR UPDATE SKIP LOCKED` and row leases support concurrency within one node and recovery after failure.

A transition can overlap the old leader's final in-flight poll and the new leader's first poll. The same at-least-once guarantees apply as for a crash during a poll. Atomic file renames or database row claims, plus idempotent queue handoffs, tolerate a repeated read without data loss. For database sources, the operator must provide atomic `poll_statement` and `mark_statement` behavior. Examples use a status flag or `UPDATE ... RETURNING` . The engine provides atomic renames only for file sources.

A partitioned leader stops within `leader_fence_timeout_seconds` . A standby waits until `leader_lease_ttl_seconds` expires. The requirement `fence < TTL` ensures the old leader stops first.

A leader that stops cleanly expires its lease, so a follower can take over immediately. After a crash or network partition, takeover waits at most `leader_lease_ttl_seconds` . Single-node behavior is unchanged: the no-op coordinator is always leader. It polls every source, does unconditional startup recovery, and starts no leader sweep.

Per-lane FIFO survives failover. Only the leader processes the graph, so nodes do not drain the same lane concurrently. After failure, a lane's first row can remain in flight under the old leader's expired lease. The ordinary FIFO claim (`claim_next_fifo`) returns this row to pending before it selects the lane head. Both actions occur in the same transaction. Thus, later rows cannot pass the recovered head. Recovery does not wait for the periodic leader sweep.

This replaces the removed active-active `lane_leases` table and per-lane ownership design. Row leases use wall-clock time and assume clock synchronization. Keep `[store].lease_ttl_seconds` above expected clock differences plus the claim interval. Single-node behavior is unchanged. SQLite and SQL Server also use one active processor.

Cross-node convergence (Track B Step 6). Nodes synchronize reference sets and configuration reloads automatically.

- **Reference sets.** Only the leader reads each external file or database source and writes the shared, versioned snapshot. Each node uses `converge_reference_cache` to update its local cache when the per-set version changes. Thus, each cluster reads the external source once per refresh. Single-node behavior is unchanged. Its coordinator always acts as leader and reads the source on each pass. SQLite needs no convergence call because its sole writer already has the current cache.
- **Configuration reloads.** A `POST /config/reload` request on one node increments the shared `cluster_config` version token. Other nodes detect the higher version and reload their own configuration directories. The initiating node updates its applied version immediately and does not reload again. A `dry_run` does not change the token. A single node does not start the convergence loop.

All nodes must have identical configuration files. The token only coordinates when each node reloads its local files. The startup sweeps for missing destinations and Handlers make the same assumption.

The implemented coordination functions include:

- Self-fencing leader election.
- Leader-only shared write tasks and source polling.
- Failover-safe per-lane FIFO recovery.
- Cross-node reference and configuration convergence.
- Transform-state read-through across nodes (Step 6b).
- The read-only `/cluster` operations API (Step 7).

A startup `INFO` entry lists the operating assumptions. Postgres and SQL Server support active-passive operation: one leader processes the graph, and a standby takes over after failure. Both recover the previous leader's in-flight rows on promotion. The old leader stops before its lease expires. The active-active design was dropped on 2026-06-18, its code was removed, and it is not planned.

Key	Type	Default	Notes
<code>enabled</code>	bool	<code>false</code>	turn on the coordination seam; requires a server-DB store (<code>[store].backend = postgres</code> or <code>sqlserver</code>) and <code>[store].pool_size >= 2</code>

Key	Type	Default	Notes
<code>node_id</code>	str	<code>unset</code>	override the auto id (<code>host:pid:hex</code>); pin for a stable identity / tests. Unset → reuses the store's lease owner-id, so node-id == owner-id
<code>heartbeat_seconds</code>	num	10	how often a node refreshes its <code>last_seen</code> heartbeat and renews its leadership lease (no separate leader-check knob). Must be > 0
<code>node_timeout_seconds</code>	num	30	a node is considered dead when its <code>last_seen</code> is older than this (the <code>/cluster/nodes</code> freshness filter). The leadership lease — not this timeout — is what transfers leadership. Must be > 0, and must exceed <code>heartbeat_seconds</code>
<code>reclaim_interval_seconds</code>	num	30	how often the leader runs the lease-reclaim sweep that recovers crashed nodes' in-flight rows (followers no-op). Must be > 0
<code>leader_lease_ttl_seconds</code>	num	30	the leadership lease TTL (active-passive self-fencing). The leader renews to <code>DB_now + this</code> ; a standby acquires only once the lease has expired (on the DB clock, so node skew is irrelevant). Must be > 0
<code>leader_fence_timeout_seconds</code>	num	20	a leader that can't renew within this (its own monotonic clock, no DB I/O) self-fences — the split-brain guard. Must be > 0, > <code>heartbeat_seconds</code> , and < <code>leader_lease_ttl_seconds</code>
<code>acquire_delay_seconds</code>	num	0	leader-preference handicap (ADR 0096, per-node). Seconds this node waits PAST the lease-expiry time before it may take over an expired lease, so a preferred (<code>0</code>) node wins the routine take-over race. NEVER delays a renewal by the current leader, and only ever makes a node claim later — so it can't open a two-leader window. Governs take-over of an expired lease only (the first election on an empty table is a plain race). Must be <code>>= 0</code> . Surfaced per-node in <code>/cluster/nodes</code>
<code>promotable</code>	bool	true	non-promotable standby flag (ADR 0096, per-node). <code>false</code> = this node may never become leader (never inserts/takes-over/renews the lease); a node that somehow already leads steps down cleanly. Use for a warm, passive DR engine. At least one promotable node must exist or no node ever acquires the lease. <code>[dr].activate</code> cannot be combined with <code>[cluster].enabled</code> (a warm DR node is a non-promotable member, not a <code>[dr]</code> box). Surfaced per-node in <code>/cluster/nodes</code>

[backup] — scheduled DR backup / restore-verify

The engine can create scheduled and on-demand disaster-recovery backups. It saves the configuration bundle and SQLite store in one AES-256-GCM `.mfbak` archive. The destination must be a local or UNC path. There is no cloud target or added network egress. See #60 and ADR 0049.

The default, `enabled = false` , disables backup operations. When enabled, the leader runs `BackupRunner` . It takes a consistent SQLite snapshot without claiming or changing staged-queue rows. It includes the loaded `--config` directory and encrypts the archive with the store DEK (ADR 0019 KeyProvider). It applies keep-N retention, does a restore verification, and records one PHI-free `dr_backup` audit row.

For Postgres or SQL Server, the database administrator provides database backups (#52). The engine makes a configuration-only backup or skips backup, as set by `config_only_on_server_db` .

Key	Type	Default	Notes
<code>enabled</code>	bool	<code>false</code>	opt-in master switch; a deployment with no <code>[backup]</code> is unaffected
<code>destination</code>	path	<code>""</code>	Local or UNC backup directory, such as <code>D:/me-for-backups</code> . Required and non-empty when backups are enabled. Cloud URLs, including <code>s3://</code> and <code>https://</code> , are rejected. No cloud target is available.
<code>schedule_at</code>	str	<code>"02:00"</code>	Daily local backup time in "HH:MM" format, using the same grammar as <code>[retention].vacuum_at</code> . An empty string disables scheduled backups. Use <code>messagefoundry backup</code> for on-demand backups.
<code>retention_keep</code>	int	7	After a new archive passes verification, delete archives older than the newest N at the destination. <code>0</code> keeps all archives. An archive that fails verification does not count toward N. A failed run cannot remove the last verified backup.
<code>snapshot_method</code>	str	<code>vacuum_into</code>	<code>vacuum_into</code> (default; takes a writer lock, sized for the off-peak schedule) or <code>online_backup</code> (low-contention, page-batched)

Key	Type	Default	Notes
<code>include_config</code>	bool	<code>true</code>	Include the loaded <code>--config</code> directory in the archive. The recovery site then has both the store and the configuration needed to use it. It need not access the organization's Git repository.
<code>verify_after_backup</code>	bool	<code>true</code>	After each backup, open the archive and run <code>integrity_check</code> and row-count checks. Enabled by default.
<code>full_restore_verify</code>	bool	<code>false</code>	Restore the snapshot to a temporary database and open it through <code>open_store</code> . This additional check is optional and runs on demand. It is not the default per-backup check.
<code>config_only_on_server_db</code>	bool	<code>true</code>	For PostgreSQL and SQL Server, the database administrator handles database backups (#52). This setting backs up the configuration bundle only. <code>false</code> skips all engine backups on a server database, including configuration archives.
<code>allow_unencrypted</code>	bool	<code>false</code>	audited escape permitting a cleartext archive on a no-key synthetic instance (the parallel of <code>[security].allow_unencrypted_phi</code>). A PHI instance with no key still refuses to write an unencrypted archive regardless of this flag

[dr] — third-tier disaster-recovery standby

The third-tier disaster-recovery engine runs high-priority feeds after the loss of the HA pair or site. It operates with reduced capacity (#61, ADR 0048). The default, `enabled = false`, disables this function.

On activation, the engine restores the store from a `[backup] .mfbak` archive. It refuses activation if the KeyProvider or DEK is unavailable at the disaster-recovery site. It starts connections at or above `priority_threshold`. Other connections report `status: "filtered"`. The engine must acquire the virtual IP address or abort activation.

Activation is manual through `POST /dr/activate` and requires `dr:operate`. Health probes cannot activate the engine. The engine reads `enabled` and `activate` at startup. The loader rejects `[dr].activate` with `[cluster].enabled`. A warm engine at the disaster-recovery site is a non-promotable cluster member, not a competing disaster-recovery engine.

Key	Type	Default	Notes
<code>enabled</code>	bool	<code>false</code>	is this deployment a DR standby box at all? <code>false</code> = the normal run-profile (every connection starts, subject only to ADR 0031), byte-unchanged
<code>activate</code>	bool	<code>false</code>	should this box come up under the DR run-profile on this boot — the startup activation latch, distinct from the runtime <code>POST /dr/activate</code> endpoint. Enabled but <code>activate = false</code> is <i>provisioned-but-passive</i> : it binds no priority feeds until an operator activates it. A no-op unless <code>enabled</code>
<code>activation_mode</code>	enum	<code>manual</code>	<code>manual</code> is the only implemented mode. An explicit, role-gated operator action activates DR. <code>auto</code> is reserved for future detection of HA-pair loss. Configuration load rejects <code>auto</code> with a "not yet supported" error.
<code>priority_threshold</code>	enum	<code>critical</code>	start only connections whose resolved priority rank is at or above this tier (<code>[delivery].priority</code> + a per-connection <code>priority=</code>). <code>critical</code> (owner-locked default) starts only the critical feeds; <code>normal</code> would also start normal-tier ones. A below-threshold connection reports <code>status: "filtered"</code> — distinct from ADR 0031's <code>"failed"</code> . An unknown value fails config load
<code>takeover_hook</code>	str	<code>""</code>	optional operator command run before binding the priority listeners: exit 0 = "VIP acquired", any non-zero or timeout = "not acquired" and activation aborts . For an ADR 0047 load-balancer topology the passive LB is the fence and this is belt-and-braces only. <code>""</code> = no hook; a whitespace-only value is rejected at load (it would run an empty shell and "succeed")
<code>release_hook</code>	str	<code>""</code>	the symmetric command run on <code>POST /dr/release</code> to hand the VIP back to the recovered primary. <code>""</code> = no hook; whitespace-only rejected at load
<code>takeover_timeout_seconds</code>	float (>0)	<code>30.0</code>	Time limit for takeover and release hooks, and for the recovery site's KeyProvider check. If a hook or key probe fails to complete within this limit, activation aborts.
<code>seed_archive</code>	path	<code>""</code>	Local or UNC <code>.mfbak</code> archive used to seed the DR store during activation. If empty, the operator supplies the path in the <code>POST /dr/activate</code> request. Cloud URLs are rejected.

Key	Type	Default	Notes
<code>restore_token</code>	path	<code>""</code>	opt-in server-DB DR restore token (BACKLOG #223, ADR 0102). A local/UNC path to a small JSON token the DBA places on the DR box recording the expected source-backup anchor of a native (Postgres/SQL Server) restore. When set, the server-DB seed gate cross-checks it against the restored database's own latest successful <code>dr_backup</code> archive — a vintage floor a bare boolean attestation cannot give: a stale or wrong native restore's latest anchor differs, so activation refuses closed . <code>""</code> (default) = off (the gate is byte-unchanged; SQLite is a no-op). A cloud URL is rejected

[approvals]

This section enables dual-control approval for high-value actions (ASVS 2.3.5). See [SECURITY.md](#). It is off by default.

When enabled, the listed operations wait for a second approver with `approvals:approve`. The requester cannot approve the same request.

Key	Type	Default	Notes
<code>enabled</code>	bool	<code>false</code>	turn on dual-control; off = every action executes inline as before
<code>operations</code>	list[str]	<code>["connection_purge", "dead_letter_replay"]</code>	which operations require approval; each must be a known op key (a typo is refused at startup)
<code>expiry_hours</code>	num	72	a pending request can no longer be approved after this many hours (<code>0</code> = never expires)

[integrity]

The engine verifies loaded first-party module files against the installed wheel's `*.dist-info/RECORD` hashes. It does this at startup and on demand (ADR 0041 D3, `messagefoundry/integrity.py`). A changed hash produces a hash-chained `startup_integrity` audit row and an `AlertSink` alert.

This detects changes to installed `site-packages` files. ADR 0036 separately protects the configuration directory. An administrator with write access to the virtual environment and restart permission can change installed files.

Attestation is on by default but only alerts. It does not block startup. An editable installation (`pip install -e .`) has no RECORD baseline, so attestation does nothing.

Key	Type	Default	Notes
<code>enabled</code>	bool	<code>true</code>	run startup attestation at all. On by default (alert-only is harmless); a no-op off an editable install. Set <code>false</code> only to suppress the check entirely (e.g. an unusual packaging where RECORD is known-stale) — you then lose the in-place-tamper tripwire.
<code>fail_closed_on_drift</code>	bool	<code>false</code>	when <code>true</code> , drift makes <code>serve</code> refuse to start (after recording the audit row + alerting). Default <code>false</code> = alert-only : a legitimate reviewed in-place security hotfix (the documented vendored-parser patch contingency) would itself trip a RECORD mismatch, so fail-closed-by-default would brick a legitimate patch. Opt in for hard enforcement on a locked-down instance.
<code>audit_verify_on_start</code>	bool	<code>false</code>	When <code>true</code> , the engine checks the complete <code>audit_log</code> hash chain once at startup (#190). A broken chain logs WARNING and sends an <code>AlertSink</code> event. It never prevents startup, because an audit fault must not cause a service outage. Disabled by default. A large audit log increases startup time.

[engine]

Not implemented. There is no `EngineSettings` model. The loader ignores an `[engine]` block without a warning because `ServiceSettings` uses `extra="ignore"`. The rows below describe proposed settings.

`data_dir` does not set a base for relative paths. For `env()` value files, use `[environments].base_dir` or `serve --project-root`. For the store, use `--db` or `[store].path`.

Key	Type	Default	Notes
<code>shutdown_timeout_seconds</code>	int	—	accepted-but-ignored (proposed): graceful stop. The ASGI lifespan's <code>engine.stop()</code> is not bounded by a setting today
<code>data_dir</code>	str	—	accepted-but-ignored (proposed): base for relative paths. Setting it anchors nothing

[service] (NSSM / Windows)

The installation settings for NSSM, including automatic restart and log paths, are in `scripts/service/`. This section controls the engine's read-only report of Windows service status (L6a, ADR 0065). The engine runs unprivileged `sc query <service_name>` outside the event loop. The console reads the result through `GET /service/status`, which requires `monitoring:read`.

The API cannot start, stop, or restart the host service. Status reporting is off by default. Set `[service]` values in the file. This section has no `MEFOR_*` environment-variable support.

Key	Type	Default	Notes
<code>report_status</code>	bool	<code>false</code>	report the run state; off = no <code>sc query</code> ever runs and the route answers <code>state = "disabled"</code>
<code>service_name</code>	str	<code>""</code>	the Windows service to query. Letters, digits, space, <code>.</code> , <code>_</code> , <code>-</code> only (anything else is refused at load); empty = disabled

[security]

The **canonical, plain-language home for the high-value security posture switches** (ADR 0118). Each switch **defaults to the secure position**; loosening one is deliberate and **warned at serve** (see `SECURITY-LOOSENING.md` for what each opt-out gives up + its ASVS/NIST/HIPAA mapping). This section **replaces** the scattered legacy keys — setting a moved key in its old section (`[api].host`, `[api].serve_ui`, `[api].public_origin`, `[auth].enabled`, `[auth].require_mfa`, `[auth].session_idle_timeout_minutes`, `[auth].session_absolute_hours`, `[store].allow_unencrypted_phi`, `[egress].deny_by_default`, `[retention].messages_days`, `[retention].allow_unbounded_phi`, `[diagnostics].audit_all_authz`, `[ai].data_class`, `[ai].production`) is **rejected at load** with a pointer to its `[security]` replacement. Low-level *plumbing* (TLS cert paths, `[egress].allowed_*` contents, `[retention].dead_letter_days`, DB identity, password policy, rate limits, AD/LDAP) stays in its functional section.

The loader maps `[security]` values to the internal fields. The startup checks and `checks.py` commit/CI checks retain their previous enforcement behavior. No existing refusal is weakened (the No-loosen rule, ADR 0092 §5). With `enforcement = enforce` (the default), PHI restrictions still fail closed regardless of values here. See ADR 0148.

key	type	default	meaning
<code>local_access_only</code>	bool	<code>true</code>	reachable only from this machine (loopback bind)
<code>listen_address</code>	str	<code>"127.0.0.1"</code>	bind address — used only when <code>local_access_only = false</code>
<code>require_encryption_for_remote</code>	bool	<code>true</code>	any off-machine access must be over TLS (config-file twin of <code>--allow-insecure-bind</code> ; can't relax production-PHI)
<code>serve_web_console</code>	bool	<code>true</code>	Mounts the browser console at <code>/ui</code> (ADR 0143). Enabled by default for local loopback binds. Set <code>false</code> for JSON-only service. On exposed instances, the default becomes JSON-only unless the console is explicitly enabled with TLS and <code>web_console_public_address</code> .
<code>web_console_public_address</code>	str	<code>""</code>	external origin when the console is exposed off-box (CSRF/CSWSH + WebAuthn RP-id)

key	type	default	meaning
allowed_client_networks	list[str]	[]	[BUILT] (ADR 0151): source-address allow-list for the operator API + web console. Empty (the default) = no restriction. Non-empty = a request whose client address is outside every listed network is refused 403 in middleware, before routing and before sign-in (also covers <code>/ui</code>, <code>/ui/static</code>, <code>/ws/stats</code>). Entries are CIDR networks or bare hosts (<code>"10.20.0.0/16"</code>, <code>"2001:db8::/48"</code>, <code>"10.20.4.7" → /32</code>), IPv4 + IPv6 mixed; malformed entries are refused at load and valid ones are stored normalized (<code>10.1.2.3/24 → 10.1.2.0/24</code>). Loopback is always allowed, with no knob (the tray <code>/health</code> poll, an on-box browser, <code>messagefoundry check</code> and a container HEALTHCHECK cannot be allow-listed). Operator surface only — the ingest listeners keep their own per-connection <code>source_ip_allowlist</code> (an attribute on the connection in <code>connections.toml</code>, not a key in the <code>[inbound]</code> section — put it there and it is silently ignored). It matches the address uvicorn reports, so it is INERT behind an UNDECLARED proxy / NAT / a bridged container — declare the proxy in <code>[api].trusted_proxies</code> or this does nothing; <code>curl /health</code> and read <code>observed_client</code> to check. Setting it tightens <code>[api].trusted_proxies</code> to single hosts (a broad range would let every host inside it forge its own source address). Startup-only: a lockout costs a service restart. Defence-in-depth behind the host firewall, not the primary network control — read OFF-LOOPBACK-DEPLOYMENT.md first. Env: <code>MEFOR_SECURITY_ALLOWED_CLIENT_NETWORKS</code> (comma-separated).
encrypt_stored_data	bool	true	PHI encrypted at rest (key from the environment)
allow_unencrypted_phi	bool	false	audited escape: start a PHI instance with no key
allow_unencrypted_phi_under_strict_enforcement	bool	false	the second acknowledgment required to start a PHI instance keyless under strict enforcement (ADR 0140). Under <code>enforcement = enforce</code>, <code>allow_unencrypted_phi = true</code> on its own is not enough — <code>serve</code> still refuses to start (exit 2) unless this is also set, so the highest-risk posture (real PHI + strict enforcement) is never one flag away from plaintext at rest. Under <code>enforcement = warn</code> the single <code>allow_unencrypted_phi</code> flag still governs. With both set the instance starts with PHI bodies, summary/metadata and the error columns unencrypted at rest, and the startup AUDIT line names both flags. A loosening — <code>security_loosenings()</code> reports it, so it is never silent

key	type	default	meaning
allow_single_factor_admin_when_exposed	bool	false	permit single-factor admin on an exposed PHI instance (ADR 0140). With <code>require_sign_in</code> on, <code>require_mfa</code> explicitly off, and the operator surface exposed (a non-loopback bind, or the console reached through a declared TLS-terminating proxy), a PHI instance under <code>enforcement = enforce</code> refuses to start (exit 2) — the Administrator role would authenticate with a single factor over the network. Setting this permits that start; it is recorded in a WARNING-level AUDIT line and the ordinary exposure warning still prints. A loosening — <code>security_loosenings()</code> reports it. Prefer <code>require_mfa = true</code>: it gates only local Administrator accounts (directory MFA is delegated), so it is safe to leave on even on an AD-only deployment
memory_encryption_operator_declared	bool	false	[BUILT] (ADR 0152 rung 2, ASVS 11.7.1): the operator's declaration that this host provides hardware memory encryption (AMD SEV-SNP / Intel TDX), so PHI is protected in RAM while it is being processed. The engine cannot verify it — a local CPU flag is emitted by the OS whose integrity the requirement protects against — so this records who took responsibility, the same discipline as <code>MEFOR_TLS_REVOCATION_ATTESTED</code>. It is deliberately not called "attested": in confidential computing that word means a CPU-signed quote verified against the silicon vendor's root PKI (ADR 0152 rung 3, not built). An exposed PHI instance without it warns and starts — on every environment, at both <code>enforcement</code> settings; it refuses only if <code>require_memory_encryption_declaration</code> is also set. A positive platform read-out does not substitute for it (a read-out must never relax a control). Loopback and synthetic instances are byte-identical (never consulted). If the platform read-out positively contradicts this, the contradiction is warned at start and reported as <code>memory_encryption_readout_contradicts_declaration</code> on <code>GET /security/posture</code> — but never refused (the read-out is a self-report, not evidence, and has known false negatives: driver not loaded, container without the device node mapped, Azure CVM paravisor). Setting this does not make the instance ASVS 11.7.1-compliant — see the read-out note below the table. Env: <code>MEFOR_SECURITY_MEMORY_ENCRYPTION_OPERATOR_DECLARED</code>

key	type	default	meaning
require_memory_encryption_declaration	bool	false	[BUILT] (ADR 0152 rung 2): turn the row-12 warning above into a refusal — an exposed PHI instance with no <code>memory_encryption_operator_declared</code> then refuses to start under <code>enforcement=enforce</code> (and still warns under <code>warn</code>). Opt-in by design, and the default is load-bearing: the property is a host property that no operator can satisfy on Windows (the read-out is always <code>null</code> there), and "exposed" includes the recommended loopback-behind-proxy topology, so a refusal by default would stop working dev/staging/prod deployments from booting on upgrade over something they cannot change. Same scoping rule as <code>[security].allowed_client_networks</code> ' companion refusal (ADR 0151): a new refusal fires only on a new opt-in. Set it in an estate that has standardized on confidential-computing hosts and wants a missing declaration to be fatal. Env: <code>MEFOR_SECURITY_REQUIRE_MEMORY_ENCRYPTION_DECLARATION</code>
require_sign_in	bool	true	authenticate every request
require_mfa	bool	true	second factor (native TOTP or a WebAuthn passkey), enforced as an access gate since ASVS 6.3.3 — an MFA-pending session is refused on <i>every</i> authorized route with <code>403 + X-MFA-Required: 1</code> , and a browser session is confined to <code>/ui/mfa</code> .
require_mfa_scope	"administrators" "every_local_account"	"every_local_account"	Which local accounts must ENROL a factor when <code>require_mfa</code> is on (ASVS 6.3.3). An account that has already enrolled one must always satisfy it, under either value — this dial only decides who is required to enrol in the first place. <code>administrators</code> restores the pre-6.3.3 posture and is reported as a loosening on <code>GET /security/posture</code> (advisory, not a refusal: refusing to boot on it would break every existing deployment on upgrade). Directory (AD/Kerberos) identities are out of scope under either value — their MFA is delegated to the directory. Operator note: under the default a non-interactive local bearer-token service account becomes MFA-pending and cannot enrol unattended — move it to mTLS (<code>require_service_cert</code> , exempt by design) or to AD, or set this to <code>administrators</code> . Env: <code>MEFOR_SECURITY_REQUIRE_MFA_SCOPE</code>
sign_out_after_idle_minutes	int	30	session idle timeout
max_session_hours	int	12	session absolute lifetime
block_unlisted_outbound	bool	true	deny-by-default egress — only allow-listed destinations send
delete_message_bodies_after_days	int	30	bounded PHI-body retention; <code>0</code> = keep indefinitely (audited)
allow_keeping_phi_indefinitely	bool	false	audited escape: unbounded PHI retention
audit_all_authorization_decisions	bool	false	ePHI access is always audited regardless; this adds full authz tracing (off by default — forcing it on risks flooding the audit log)

key	type	default	meaning
<code>handles_real_patient_data</code>	bool	<i>derived</i>	the master data-class lever (was <code>[ai].data_class = "phi"</code>). Unset $\Rightarrow$ derived from the environment name — all three built-in names (<code>dev</code> / <code>staging</code> / <code>prod</code>) now derive PHI (ADR 0148 GIVEN 1, so the default/CI path exercises the encryption/egress/retention controls rather than first meeting them in production); a genuinely-synthetic dev/CI box must set <code>false</code> explicitly (a loud, audited opt-out), and a custom-named env must declare it
<code>enforcement</code>	<code>enforce</code> <code>warn</code>	<code>enforce</code>	the serve-gate refuse/warn dial + the ADR 0092 escape-clamp key (ADR 0148 GIVEN 2). <code>enforce</code> (default) refuses every PHI serve-gate violation and shuts every blunt escape-clamp — byte-identical to the former production-tier behaviour; <code>warn</code> logs + audits + continues and honours the escapes (a loud, audited loosening, named by <code>security_loosenings()</code>). Decoupled from <code>production_instance</code> (env <code>MEFOR_SECURITY_ENFORCEMENT</code>)
<code>production_instance</code>	bool	<i>derived</i>	production-tier posture (was <code>[ai].production</code>). Derived from the environment name when unset (<code>prod</code> $\rightarrow$ yes; <code>dev</code> / <code>staging</code> $\rightarrow$ no). Informational since ADR 0148 — drives the AI data-scope ceiling, the DEBUG-log refusal, and reporting, not the serve-gate refuse/warn dial (that is <code>enforcement</code>)

Editing is IDE-only: the VS Code extension's *Edit Security Settings* command (which shells `messagefoundry security show|set`) is the sole authoring surface. The **web console is read-only** — the effective posture, active loosening, and the synthetic-relaxation notice are surfaced at `GET /security/posture` (authenticated, `monitoring:read`). Authentication & RBAC *plumbing* remains in `[auth]` (see SECURITY.md); the at-rest-encryption *key* is a secret supplied via `MEFOR_STORE_ENCRYPTION_KEY` / `[store].encryption_key_file` (PHI.md).

The engine reads this section at startup. Restart the engine to apply a changed switch. `POST /config/reload` reloads the `--config` graph, not `[security]`.

The memory-encryption read-out is *not* a compliance claim

The `GET /security/posture` response includes platform information beside the FIPS attestation. The report-only fields are `memory_encryption_self_reported_capability`, `memory_encryption_self_reported_active`, `memory_encryption_self_reported_mechanism`, and `memory_encryption_readout_source`. See ADR 0152, rung 1.

On Linux, `/proc/cpuinfo` flags report capability. The `/dev/sev-guest` and `/dev/tdx-guest` device nodes indicate guest activation. Capability and activation are separate fields. The engine does not derive one from the other. On Windows, all fields are `null` because the in-guest attestation path is not implemented.

No value of any of these fields satisfies ASVS 11.7.1, and none may be cited as though it did. The host operating system reports these values about itself. A compromised kernel or hypervisor can forge them. Evidence would require a CPU-signed attestation report verified against the silicon vendor's root PKI. That verification is ADR 0152 rung 3 and is not implemented. Use this report to confirm configuration, not as proof.

Each response includes `memory_encryption_note`, which contains the same disclaimer as the startup warning. The disclaimer thus remains with copies of the response. The `memory_encryption_operator_declared` field records an operator declaration, not an attestation.

The `memory_encryption_readout_contradicts_declaration` field has three states. A `null` value means the engine measured nothing that could contradict a declaration. This applies on Windows, on AMD SME or Intel TME hosts without a guest interface, and in containers without the device node. It also applies when no declaration exists. A `false` value means the measured report agrees with the declaration. The engine does not report agreement without a measurement.

The property itself is a host requirement, not a switch. `memory_encryption_operator_declared = true` records a claim; it does not create memory encryption. An ASVS **Level 3** PHI deployment must actually run the engine as a **confidential guest** on an AMD SEV-SNP or Intel TDX host — [SYSTEM-REQUIREMENTS.md](#) states the requirement and the (verified) availability picture, which today is **not reachable for a Windows guest on on-premises Hyper-V or ESXi**. If the host lacks this property, leave the declaration unset. Keep the startup warning and disclose ASVS 11.7.1 as **Partial**. Do not change `[security].enforcement` to `warn` for this purpose. That setting also weakens other posture refusals. This control refuses startup only if you enable `require_memory_encryption_declaration`. For instructions, see [OFF-LOOPBACK-DEPLOYMENT.md](#), In-use data protection.

Example

```
# messagefoundry.toml
[store]
backend = "sqlserver"
server = "sql01.hospital.local"
database = "MessageFoundry"
auth = "sql"
username = "mefor_service"
encrypt = true

[security]
local_access_only = true           # loopback bind (ADR 0118; the bind host lives here, not [api])
delete_message_bodies_after_days = 30 # the PHI-body window (ADR 0118; NOT [retention].messages_days)

[api]
port = 8765

[logging]
level = "info"
format = "json"                # structured stdout (one JSON object per line)
# Setting forward_host turns forwarding ON by default (ADR 0080); forward_enabled = false opts out.
forward_host = "siem.hospital.local" # ship a copy off-box to a syslog/SIEM collector
forward_port = 6514                # RFC 5425 syslog-over-TLS default
forward_protocol = "tls"           # udp (default) | tcp | tls (native RFC 5425, no agent)
forward_tls_ca_file = "C:/mefor/siem-ca.pem" # required for tls unless forward_tls_verify = false
# Opt-in startup clock-sync gate (ASVS 16.2.2) – warns on skew; add fail-closed to refuse start:
# require_time_sync = true
# ntp_peer = "ntp.hospital.local"

[retention]
# The inbound-body window is [security].delete_message_bodies_after_days above – setting
# messages_days here is REJECTED at load (ADR 0118). Only the plumbing keys stay in this section:
dead_letter_days = 90             # null dead-letter bodies after 90 days
vacuum_at = "03:30"               # daily off-peak VACUUM to reclaim space (SQLite only)
```

```
# secret via env (never in the file)
set MEFOR_STORE_PASSWORD=...
```

Build order (incremental)

1. **Done** — `ServiceSettings` model + loader (file + env + CLI precedence); `[api]` / `[logging]` and `[store]` `backend=sqlite|path|synchronous` wired into `serve` (`--service-config` + the `--db` / `--host` / `--port` / `--log-level` overrides).
2. **Done** — `[delivery]` defaults feed the default `RetryPolicy` (`DeliverySettings.retry_policy()`), alongside the ordering / internal-error / buildup-stall-saturation / priority defaults an outbound inherits when it declares none.
3. **Done** — `[store]` server-DB keys landed with the SQL Server **and** Postgres backends (both consume `server` / `database` / `username` / `pool_size` / `command_timeout` / `ssl_root_cert`); they are **implemented**, not accepted-but-ignored.
4. **Done** — `[retention]` purge/maintenance job (body-null + WAL/VACUUM, audited; `audit_days` reserved). `[logging]` structured-JSON `format` + off-box `forward_*` syslog shipping land (sec-offbox-log); PHI redaction is an always-on handler filter (no structlog).

Decisions and open questions

-  **Decided and built** — **TOML file + env + CLI** as above, chosen for consistency with `pyproject.toml` and ops-friendliness; secrets via `env`.
-  **Settled** — **the IDE authors, the web console reads**. Settings are **edited from the VS Code extension** (its *Edit Security Settings* command shells `messagefoundry security show|set`, which owns the TOML write) or by hand in `messagefoundry.toml`; there is **no settings-write API**. The web console is **read-only** on posture — `GET /security/posture` (authenticated, `monitoring:read`) surfaces the effective switches and active loosening. This is the reverse of the split originally sketched here.
- Whether per-connection overrides (e.g. a connection's own retry) stay in code (today) or also move into settings. Recommendation: **keep per-connection logic in code**, service settings are defaults.

© 2026 **MessageFoundry, LLC** · MessageFoundry is licensed under **AGPL-3.0-or-later** · Document generated September 2026 for MessageFoundry v0.3.2.

MessageFoundry and the MessageFoundry word mark are trademarks of MessageFoundry, LLC. This document describes an **Early Access** release; check pypi.org/project/messagefoundry for the current version. Verify specifics against your own deployment and the engine's reference documentation.